



SAPIENZA
UNIVERSITÀ DI ROMA

Integrating Zero-Knowledge in SCAPI

Facoltà di Ingegneria dell'Informazione, Informatica e Statistica
Dipartimento d'Informatica
Corso di Laurea in Informatica

Candidato
Ilaria Scarselli
Matricola 1918975

Relatore
Daniele Venturi

Anno Accademico 2022/2023

Tesi discussa il giorno 11 Dicembre 2023

di fronte a una commissione esaminatrice composta da:

Prof. Friolo, Prof. Panizzi, Prof. Quattrocioni, Prof. Salvo, Prof. Scardapane,
Prof. Venturi, Prof. Mei (presidente)

Integrating Zero-Knowledge in SCAP

Tesi di Laurea. Sapienza – Università di Roma

© 2023 Ilaria Scarselli. Tutti i diritti riservati

Questa tesi è stata composta con L^AT_EX e la classe Sapthesis.

Email dell'autore: ilascarselli@gmail.com

Indice

1	Introduzione	1
1.1	Obiettivi del tirocinio	1
1.2	Struttura della tesi	2
2	Protocolli Interattivi e Zero-Knowledge	3
2.1	Introduzione ai protocolli considerati	3
2.1.1	Protocolli Non-Interattivi	3
2.1.2	Protocolli Interattivi	4
2.2	Zero-Knowledge	8
2.2.1	Zero Knowledge a numero di round non costante	9
2.2.2	Zero Knowledge a numero di round costante	10
2.2.3	Honest-Verifier Zero-Knowledge	11
2.2.4	Non-Interactive Zero-Knowledge	12
2.2.5	Sigma-Protocols	13
2.3	Predictable Arguments of Knowledge	14
2.3.1	PAoK laconici	15
2.3.2	PAoK e Extractable Witness Encryption	16
2.3.3	PAoK e Zero-Knowledge	16
2.3.4	Predictable ZAPs	17
2.3.5	Commit-and-Prove Predictable Arguments	17
3	Panoramica su SCAPI	18
3.1	Architettura di SCAPI	19
3.1.1	Communication Layer	19
3.1.2	Layer 1: Basic Primitives	20
3.1.3	Layer 2: Non-Interactive Schemes	21
3.1.4	Layer 3: Interactive Protocols	21
3.2	Utilizzo nel progetto di tirocinio	21
4	Implementazione di un protocollo Zero-Knowledge	23
4.1	Decisional Diffie-Hellman assumption	23
4.2	Scelte e prove preliminari	25
4.3	Implementazione	26
4.3.1	Creazione del Thread di esecuzione	28
4.3.2	Prover	29
4.3.3	Verifier	30
4.3.4	Osservazioni finali	32
5	Conclusioni	33

Appendice	35
Bibliografia	40
Ringraziamenti	42

Lista degli Algoritmi

1	Non Isomorfismo tra grafi con private-coin	6
2	Descrizione del protocollo di Goldreich-Kahan	11
3	Simulatore nel caso di HVZK	12
4	Schema generico della fase di interazione	14
5	Generico protocollo PAoK	15
6	Chaum-Pedersen ZKP per DDH	24

Elenco delle figure

2.1	Schema di un generico protocollo non-interattivo	4
2.2	Schema di un generico scambio di messaggi interattivo	4
3.1	Rappresentazione grafica dei layers	19
3.2	Esempio di gerarchia di livelli di sicurezza in SCAP	20
4.1	Fasi seguite nell'implementazione [4]	25
4.2	Gerarchia di messaggi [5]	28
5.1	Diagramma delle classi	38
5.2	Gerarchia delle classi generata con la documentazione Javadoc	39

Capitolo 1

Introduzione

I Predictable Arguments of Knowledge (PAoK) e i Protocolli Zero-Knowledge rappresentano un'innovazione significativa nell'ambito della sicurezza informatica e della crittografia moderna, emergendo come risposta alla crescente esigenza di dimostrare la conoscenza di informazioni sensibili senza divulgarle direttamente e senza, quindi, compromettere la privacy.

Gli Zero-Knowledge Protocols, o Zero-Knowledge Proofs (ZKP), consentono a un prover di dimostrare la veridicità di una dichiarazione senza rivelare alcuna informazione oltre al fatto che la dichiarazione sia vera. In altre parole, il prover dimostra di possedere una certa conoscenza senza doverla condividere con il verifier o con qualsiasi altra parte. Questa proprietà è particolarmente utile in scenari in cui privacy e sicurezza sono cruciali, in quanto gli ZKP forniscono una forma molto forte di riservatezza. Un esempio classico è il problema della colorazione dei grafi: un prover può dimostrare di sapere come colorare un grafo in modo che nessun nodo abbia lo stesso colore di un nodo adiacente senza rivelare effettivamente la colorazione.

I Predictable Arguments of Knowledge (PAoK) introducono un elemento di predittività che consente al verifier di acquisire una certa fiducia sulla validità della dimostrazione del prover, senza conoscere l'informazione segreta vera e propria. Questi argomenti sono impiegati in contesti di crittografia e dimostrazioni di conoscenza, in cui consentono alle due parti di interagire tra loro in modo che chi ha il compito di verificare possa fare predizioni sulle risposte e sul comportamento dell'altro, senza apprendere la conoscenza effettiva.

Entrambi questi concetti trovano applicazioni pratiche significative e la loro implementazione apre la strada a nuove soluzioni innovative nei campi della sicurezza informatica e della crittografia, fornendo mezzi efficaci per dimostrare la conoscenza senza compromettere la riservatezza. Ad esempio, nei protocolli di voto elettronico, un elettore potrebbe dimostrare di aver votato correttamente senza rivelare il proprio voto specifico. Nelle transazioni finanziarie online, un utente potrebbe dimostrare di avere fondi sufficienti senza rivelare l'importo esatto o altri dettagli sensibili.

1.1 Obiettivi del tirocinio

Il tirocinio si è articolato principalmente in due fasi.

In un primo momento, è stata dedicata particolare attenzione allo studio approfondito dei protocolli appena presentati, affiancati dall'esplorazione di aree tematiche correlate, con l'obiettivo di raggiungere una comprensione completa di tali argomenti. Successivamente, la seconda fase del tirocinio è stata focalizzata sull'implementazione

dell'ipotesi decisionale Diffie-Hellman (Decisional Diffie-Hellman assumption, DDH) in Java. Questa implementazione è stata realizzata mediante l'utilizzo di interfacce della libreria SCAPI. Questa dualità di approccio ha contribuito a fornire una visione completa, con una componente pratica che ha consolidato la comprensione attraverso l'effettiva implementazione dei concetti studiati.

1.2 Struttura della tesi

La tesi è strutturata in tre capitoli distinti. Il primo capitolo si focalizza sulla discussione dei concetti teorici, inquadrandoli attraverso le loro connessioni e correlazioni. Successivamente, nel secondo capitolo, viene presentata la libreria esterna utilizzata per l'implementazione, offrendo una panoramica delle sue caratteristiche e componenti chiave. Il terzo capitolo costituisce una descrizione approfondita dell'implementazione stessa. In questa sezione, si analizza nel dettaglio il problema considerato, esaminando le parti implementate e illustrandone le scelte progettuali adottate. L'elaborato si conclude infine con un capitolo volto a sintetizzare i principali risultati ottenuti e le riflessioni emerse durante il percorso di studio e di implementazione. In queste pagine finali, vengono evidenziate le implicazioni più ampie degli approfondimenti svolti, oltre a inquadrare ulteriormente la tipologia di protocolli considerati nel contesto di possibile utilizzo attuale e a suggerire possibili direzioni per ricerche future. La conclusione offre, quindi, una prospettiva globale sull'importanza e l'applicabilità pratica, chiudendo in modo significativo il lavoro svolto.

Capitolo 2

Protocolli Interattivi e Zero-Knowledge

2.1 Introduzione ai protocolli considerati

I protocolli crittografici sono strumenti fondamentali per garantire la sicurezza delle comunicazioni digitali. I loro ruoli possono essere diversi a seconda dell'obiettivo che si vuole raggiungere, che può spaziare dall'assicurare la confidenzialità delle informazioni, al verificare la fonte di quest'ultime, o all'assicurarne l'integrità o la non-ripudiabilità, potendo talvolta anche mirare a una combinazione di queste caratteristiche. Tra questi protocolli, esiste una categoria che coinvolge dimostrazioni zero-knowledge in cui emerge una distinzione cruciale tra protocolli interattivi e non-interattivi, ciascuno con qualità che li rendono adatti a specifici contesti di utilizzo e requisiti di sicurezza. Mentre quelli non-interattivi offrono vantaggi in termini di efficienza e privacy, i protocolli interattivi sono fondamentali in situazioni in cui l'adattabilità dinamica è essenziale per il successo della comunicazione sicura.

2.1.1 Protocolli Non-Interattivi

I protocolli non interattivi consentono a una delle parti coinvolte di dimostrare la conoscenza di un segreto senza dover interagire direttamente con la controparte. Questi protocolli risultano particolarmente utili in scenari in cui l'interazione diretta tra le parti non è praticabile o desiderabile. In questo contesto, i passaggi generalmente coinvolgono un setup iniziale, durante il quale vengono inizializzati i parametri e le informazioni condivise. Successivamente, il prover, chiamato anche Peggy nell'ambito crittografico (in sostituzione dell'Alice dei protocolli classici), invia un singolo messaggio contenente una prova del segreto che desidera dimostrare di possedere. Il verifier, o Victor (assumendo il ruolo di Bob), lo analizza tramite un algoritmo di verifica e decide se accettare o rifiutare il risultato fornito dall'altra parte, senza richiedere ulteriori informazioni o interazioni. Questo procedimento può essere riassunto e visualizzato con l'immagine seguente [4].

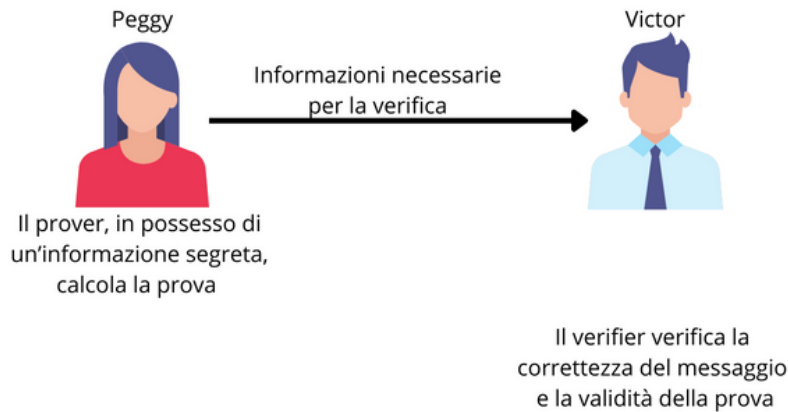


Figura 2.1. Schema di un generico protocollo non-interattivo

2.1.2 Protocolli Interattivi

I protocolli interattivi, d'altra parte, coinvolgono le parti in un dialogo continuo e attivo, consentendo a prover e verifier di adattarsi dinamicamente alle informazioni scambiate di volta in volta durante il processo. Un esempio comune è lo scambio di chiavi Diffie-Hellman, in cui le due parti possono accordarsi su una chiave segreta senza doverla trasmettere direttamente attraverso un canale di comunicazione potenzialmente non sicuro. Questo scambio può essere considerato come facente parte di questa categoria in quanto, dopo una fase di setup iniziale, vengono scambiati due messaggi: uno contenente la chiave pubblica del primo partecipante e un'altro con quella del secondo. Venendo a conoscenza delle informazioni derivanti da questo scambio, ognuna delle due parti può calcolare autonomamente il valore della chiave condivisa.

In generale, a seconda dei contesti, il verifier potrebbe formulare richieste di informazioni aggiuntive per il prover o potrebbero essere presenti anche più fasi di challenge e risposta, seguendo uno schema di tipo [4]:

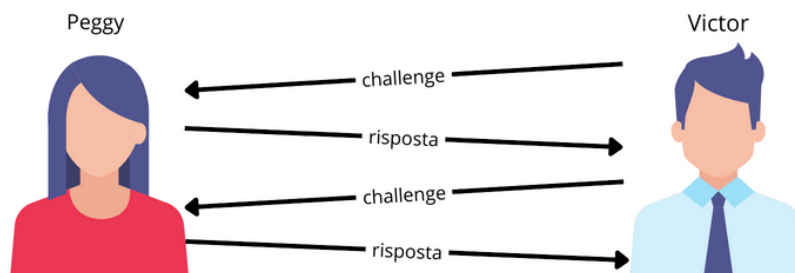


Figura 2.2. Schema di un generico scambio di messaggi interattivo

Tipicamente i problemi che si potrebbero voler risolvere in un sistema a dimostrazione interattiva possono essere suddivisi in due categorie:

- problemi di decisione, o decisionali, in cui viene preso in considerazione un'insieme di numeri o di stringhe e l'obiettivo è quello di stabilire se una determinata istanza del problema appartiene o meno a quell'insieme. Più in generale, avendo uno statement x , il compito del prover è quello di dimostrare e convincere il verifier che $x \in L$, dove L è il linguaggio considerato.

- problemi di funzione, rappresentati da una funzione $f : \mathbb{N} \rightarrow \mathbb{N}$ la cui istanza del problema è un x , mentre la soluzione è il valore di $f(x)$. L'obiettivo del prover può essere quello di cercare di dimostrare di conoscere il valore corretto di $f(x)$ senza rivelare x stesso.

La validità di un sistema a dimostrazione interattiva è vincolata a requisiti specifici:

- Completezza (Completeness): nel caso di Perfect Completeness, il sistema deve accettare sempre la dimostrazione di un'affermazione vera (con probabilità 1). Se ci sono algoritmi probabilistici che consentono quindi un certo margine di errore, è il caso della normale completezza.
- Correttezza (Soundness): la probabilità che un Prover non onesto riesca a convincere il Verifier pur non conoscendo la "witness" corretta è molto bassa⁰. Si può parlare di Perfect Soundness quando questa probabilità è esattamente zero.

Uno scenario in cui l'interazione si rivela particolarmente utile è rappresentato dalla classe di complessità computazionale NP (*Nondeterministic Polynomial Time*, da cui l'abbreviazione), in cui si può avere un verificatore deterministico. Questa classe comprende problemi decisionali per i quali è possibile trovare una soluzione in tempo polinomiale utilizzando una macchina di Turing¹ non deterministica. La peculiarità di NP è che, se si possiede una soluzione candidata, la sua correttezza può essere facilmente verificata. Questa caratteristica è ciò che rende l'interazione particolarmente adatta in questi contesti. Le dimostrazioni si fondano su problemi che sarebbero di difficile risoluzione per un Prover che non è in possesso di una specifica "witness" per l'istanza del problema in questione, ma che al contrario, risultano facilmente verificabili per il Verifier che deve decidere se accettare o rifiutare quanto affermato dalla controparte. Entrambi i partecipanti possono mantenere e aggiornare lo stato corrente durante la comunicazione, facendo in modo che i messaggi scambiati possano essere visti come una funzione di tutti gli altri messaggi inviati e ricevuti fino a quel momento.

Il vero potenziale dei sistemi a dimostrazione interattiva è ottenuto quando si permette al verifier di essere probabilistico, lasciandogli un piccolo margine di errore che è possibile manipolare (in particolare cercando di ridurlo) mediante l'uso congiunto di ripetizioni e randomizzazione. L'elemento randomico è attribuito al verifier, che possiede una moneta virtuale² che è mantenuta segreta e, per questo motivo, le risposte del prover rimangono solo in funzione dei messaggi inviati dalla controparte [AB07]. Il probabilismo del verifier porta alla definizione di una nuova classe di complessità, IP [SH92], nel cui caso le definizioni di Completezza e Correttezza vanno riformulate:

- Completezza: se $x \in L$, allora esiste un Prover P per cui la probabilità di riuscire a convincere il Verificatore V è almeno $2/3$.

⁰Le proprietà qui presentate verranno dettagliate in maniera più approfondita nelle sottosezioni dedicate agli ZKP e ai PAoK di seguito espone.

¹Una macchina di Turing è un modello astratto di calcolatore, che può essere visto come un particolare tipo di automa, ideato per rappresentare un sistema di calcolo formale, costituito da un nastro infinito suddiviso in celle, una testina di lettura/scrittura che può spostarsi avanti e indietro sul nastro e uno stato interno che varia in risposta alle istruzioni. La macchina di Turing opera seguendo un insieme di regole definite che specificano come leggere, scrivere o muovere la testina, e cambiare lo stato interno in base allo stato corrente e al simbolo letto sul nastro. Di fondamentale importanza nello sviluppo della teoria della computazione e nel comprendere le capacità e le limitazioni dei sistemi di calcolo.

²Si confronti la terminologia contenuta nell'appendice posta a conclusione di questa trattazione.

- Correttezza: se $x \notin L$, allora per ogni Prover P , la sua probabilità di riuscire a convincere un Verifier V è di al più $1/3$.

E' utile considerare un'ulteriore assunzione teorica, ovvero l'avere un Prover P con potenza di calcolo illimitata, ad esempio anche garantendogli l'accesso a un Oracolo che lo aiuti nel procedimento di prova fornendogli alcune informazioni specifiche sul linguaggio, mantenendo invece il Verifier V limitato.

E' infine anche possibile definire prove interattive che coinvolgono più partecipanti, in particolare più provers. In questo caso, l'insieme di linguaggi considerato è il **MIP**, acronimo di *Multiprover Interactive Proofs* e l'assunzione fondamentale che si intende far rispettare è che i provers possano comunicare tra loro prima dell'inizio dell'interazione con il verificatore, eventualmente stabilendo anche una strategia condivisa per rispondere alle domande, ma non durante l'esecuzione del protocollo.

Problema del non isomorfismo tra grafi

Uno dei problemi in IP , ma non noto per essere anche in NP , è il non isomorfismo tra grafi. Due grafi G_1 e G_2 si dicono isomorfi ($G_1 \equiv G_2$) se possono essere considerati uguali a meno della numerazione dei vertici, ovvero se esiste una corrispondenza uno-a-uno tra i loro vertici che conserva la connettività degli archi e se esiste una permutazione π delle etichette assegnate ai nodi di G_1 tale che $\pi(G_1) = G_2$. La formulazione del problema GI (Graph Isomorphism) è la seguente: date le rappresentazioni di due grafi, stabilire se questi siano isomorfi tra loro. $GI \in NP$, in quanto il certificato per una particolare istanza del problema è la permutazione π stessa. Per il problema GNI , complementare a GI , ovvero determinare se due grafi non godono della proprietà di isomorfismo esiste invece un sistema di prova interattiva:

Algorithm 1 Non Isomorfismo tra grafi con private-coin

- 1: Victor: sceglie randomicamente $i \in \{1, 2\}$ e permuta sempre randomicamente i nodi di G_i ottenendo un nuovo grafo H da mandare a Peggy.
 - 2: Peggy: elabora quale tra G_1 e G_2 è stato usato per produrre H . Denominando G_j quel grafo, manda l'indice j a Victor.
 - 3: Victor: accetta se e solo se $i = j$, altrimenti rifiuta.
-

Si noti come se i due grafi non sono isomorfi, allora esiste P con illimitata potenza di calcolo che riuscirà sicuramente a convincere V , perchè riuscirà sicuramente a trovare il grafo corretto. Questo non può essere vero se i due grafi sono invece isomorfi, in quanto può solamente tirare a indovinare e in questo caso la probabilità si riduce ad al più $1/2$, che può ulteriormente essere ridotta per arrivare a $1/3$ per soddisfare la proprietà di correttezza attraverso ripetizioni sequenziali o parallele (in realtà può essere ridotta anche ulteriormente con un numero sufficientemente grande di ripetizioni).

Oltre ai successivi protocolli Zero-Knowledge (ZKP) e Predictable Arguments of Knowledge (PAoK) che saranno esposti, meritano menzione due concetti di particolare rilevanza.

Arthur-Merlin

Risulta di interesse citare AM^3 , una classe di linguaggi molto simile a IP, ma che ne limita il numero dei round a un certo valore costante k , ottenendo una classe distinta per ogni $AM[k]$. Un'ulteriore differenza è che si tratta di protocolli public-coin, dove la randomicità di V (Arthur) viene condivisa con P (Merlin) includendo nei messaggi scambiati anche i valori dei lanci delle monete di V. Si definisce AM come la classe $AM[2]$, le cui prove interattive consistono nella seguente sequenza dei messaggi:

- V: manda una stringa randomica;
- P: risponde con un messaggio;
- la decisione di accettare o meno è ottenuta applicando una funzione in tempo deterministicamente polinomiale al trascritto dello scambio.

$AM[k]$ con $k > 2$ può essere riportato in $AM[2]$ permettendo a V di effettuare più lanci per poi includerli tutti nel messaggio da inviare a P. E' stato dimostrato che $IP[k] \subseteq AM[k+2]$, il che rende evidente come il permettere al prover di avere accesso ai risultati delle monete randomiche di V non tolga al Verifier stesso tanto potere quanto ci si potrebbe aspettare. La classe MA è la classe corrispondente ad $AM[2]$ in cui è il prover a inviare il primo messaggio.

Si può notare come di fatto nella **Tabella 2.2** ci sia un solo messaggio scambiato.

Arthur	Merlin
Invio di una stringa	
	Ricezione della stringa e invio di una risposta

Tabella 2.1. Protocollo in AM.

Arthur	Merlin
	Invio del certificato
lancio dei dadi e applicazione della funzione di verifica	

Tabella 2.2. Protocollo in MA.

Oblivious Transfer

L'Oblivious Transfer è un protocollo crittografico a due parti che consente a una parte, chiamata Mittente (o Sender) S, di inviare una tra potenzialmente molte informazioni in suo possesso ad un'altra parte, chiamata Destinatario (o Receiver) R, senza che il Mittente sappia quale messaggio è stato effettivamente inviato. L'obiettivo principale è garantire la privacy. Esistono due varianti principali di OT:

- 1-out-of-2 OT: Il Mittente ha due messaggi, e il Destinatario può riceverne solo uno. S non sa quale messaggio R riceve. Allo stesso tempo, R ha un bit b e vorrebbe ricevere il messaggio m_b senza che S venga a conoscenza di questa informazione.
- 1-out-of- n e k -out-of- n OT: il primo è una generalizzazione di 1-out-of-2 OT. Il Mittente ha n messaggi e vuole far sì che il Destinatario R ne riceva solo

³L'analogia è con i famosi personaggi medievali: Merlin, con la sua magia, incarna il potere computazionale illimitato del Prover, ma sebbene re Arthur, qui il Verifier, non ha modo di nascondere alla magia del mago i risultati dei suoi lanci di monete passati, Merlin non può predire i lanci futuri [BM88, AB07].

uno, mentre il destinatario possiede un indice i e vuole ricevere l' i -esimo messaggio tra gli n , senza comunicare l'indice i . Su questo si basa l'ulteriore generalizzazione k -out-of- n , in cui R vuole riceverne k , dove k è minore o uguale a n , che possono essere ottenuti contemporaneamente oppure tramite richieste sequenziali.

Il protocollo più basilare funziona in questo modo:

- Fase di Scelta: quando riceve in input (**receive**, sid , b) da R , dove b è 0 o 1, se non sono presenti messaggi di questa forma memorizzati, il protocollo memorizza il messaggio appena ricevuto e invia (**receive**, sid) a S .
- Fase di Trasferimento: con input (**send**, sid , x^0 , x^1) da S , con x^0 , x^1 appartenenti a $\{0, 1\}^k$, se messaggi di questa forma non sono memorizzati e c'è un messaggio (**receive**, sid , b) presente in attesa, il protocollo invia un messaggio (**sent**, sid , x^b) a R , dove b è il bit precedentemente ricevuto nella fase di scelta.

In questo modo, il protocollo facilita la comunicazione tra due entità, R e S , permettendo loro di condividere informazioni senza la necessità di conoscere reciprocamente il contenuto specifico degli input, cosa ampiamente utilizzata in vari contesti, come ad esempio nei protocolli di voto elettronico sicuro.

2.2 Zero-Knowledge

Mentre per i protocolli interattivi possiamo individuare radici più antiche con il protocollo di scambio di chiavi Diffie-Hellman del 1976, il concetto di "zero-knowledge proof" è stato introdotto da Shafi Goldwasser, Silvio Micali e Charles Rackoff nel 1985 [GMR85] e ha rivoluzionato il modo in cui è possibile garantire la sicurezza delle informazioni aprendo la strada a una serie di applicazioni.

Sebbene sia possibile individuarne diversi tipi e diverse caratterizzazioni, che sono andate nel tempo ad arricchire l'ambito delle dimostrazioni a conoscenza zero, si può notare come uno dei punti fondamentali che accomuna tutte le varianti è l'importanza della randomicità di alcuni passaggi o procedimenti gestiti dal Verifier, primo tra tutti quello della creazione della challenge.

La simulazione è uno strumento importante che aiuta a dettagliare a livello di teoria computazionale le caratteristiche e il comportamento di un Prover P e di un Verifier V in una prova a conoscenza zero. Dopo aver trattato di come, in generale, per avere un algoritmo interattivo corretto un prover disonesto P^* ⁴ abbia al più una certa probabilità fissata di convincere un verifier, nei contesti ZK si considera in particolare che sia anche V^* a poter agire non necessariamente in maniera conforme al protocollo, cercando pertanto di scoprire informazioni aggiuntive dal prover.

Prima di dare una definizione di Zero-Knowledge, si consideri una macchina polinomiale probabilistica A . Si indica con $A(x, y, r)$ il suo output con input x , input ausiliario y e nastro casuale r ⁵. La macchina lavora in tempo polinomiale rispetto a $|x|$. Considerando ora una seconda macchina B , $\text{output}_B(A(x, y, r_A), B(x, z, r_B))$ è l'output di B in seguito all'interazione con A su input comune x e con i rispettivi nastri casuali, a cui si aggiunge, rispetto alla notazione precedentemente riportata,

⁴Dove il simbolo dell'asterisco indica, in queste situazioni, che non ci sono garanzie circa l'onestà del partecipante a cui è associato, e pertanto suggerisce che potrebbe al contrario agire in modo disonesto o malevolo.

⁵Il nastro casuale rappresenta nel contesto teorico relativo agli automi la fonte di randomicità, con un certo numero di bit casuali usati su input x .

z , che è l'input ausiliario di B . Se i nastri r_A e r_B vengono omessi dalla notazione, si intendono non fissati, ma scelti in maniera casuale. Possiamo far assumere ai due automi i ruoli di P e V , che ricoprono le stesse classiche posizioni e, in particolare, P , utilizzando come aiuto il proprio input ausiliario privato, cerca di convincere V dell'appartenenza di x a un certo linguaggio L . Il sistema rispetta anche in questo caso proprietà di completeness e soundness:

- Completezza: quando due macchine oneste, P e V , interagiscono su un input comune $x \in L$, V sarà convinto della correttezza dello statement, a meno di una probabilità al più trascurabile.
- Correttezza: quando V interagisce con un Prover P^* su un input comune $x \notin L$, V sarà convinto ad accettarne la correttezza con probabilità al più trascurabile.

L'obiettivo generale di qualsiasi tipo di dimostrazione a conoscenza zero è consentire al prover di attestare la validità di una certa affermazione o di provare di essere a conoscenza di una certa informazione, senza compromettere la riservatezza degli elementi coinvolti diffondendo dettagli che potrebbero comprometterne la privacy o la sicurezza.

Simulazione per la black-box ZK

Il termine "simulatore" si riferisce a un dispositivo teorico o a un software che cerca di imitare o riprodurre il funzionamento di un sistema in un ambiente virtuale. L'obiettivo principale di un simulatore è fornire una rappresentazione accurata. E' uno strumento fondamentale per dimostrare la robustezza e la sicurezza dei protocolli a conoscenza zero nel preservare la privacy del Prover. Mostrando che un osservatore esterno non riuscirebbe a distinguere tra una vera dimostrazione e una simulazione della stessa, si verifica il raggiungimento, da parte del protocollo, degli obiettivi di sicurezza desiderati.

Una prova è detta a conoscenza zero se esiste un simulatore che può generare la vista o in particolare l'output del Verifier V^* partendo da uno statement. Il caso specifico di black-box ZK, si ottiene considerando V^* come una scatola nera, a cui il Simulatore S può rivolgersi esclusivamente tramite accesso a oracolo [Lin21]. Questo vuol dire che S non ha accesso diretto alle informazioni interne di V^* , ma può solamente fare interrogazioni all'oracolo (in questo caso quindi il verificatore) su come risponderebbe a determinate richieste in specifiche circostanze, senza ottenere alcun dettaglio sul processo interno di verifica o sulla conoscenza posseduta, in modo da non apprendere nulla durante il processo di simulazione, preservando la sicurezza della dimostrazione.

2.2.1 Zero Knowledge a numero di round non costante

Quando non si impone un numero costante di round, il verifier può continuare a fare ulteriori richieste sequenziali al prover, il cui numero effettivo può dipendere dal soddisfacimento di certe condizioni, dalle decisioni prese durante l'esecuzione del protocollo, o dalle risposte di P . Un round rappresenta una singola iterazione di scambio di messaggi tra le parti coinvolte.

Ad esempio, è possibile considerare il problema della 3-colorazione di un grafo: il prover vuole dimostrare di riuscire a creare colorazioni a tre colori valide di un grafo, senza scoprire tutta la colorazione stessa al verifier. All'inizio del protocollo, P^* esegue un commitment (si veda la terminologia nell'Appendice A) nei confronti di

una colorazione e la challenge del verifier consiste nel chiedere di aprire i commitment relativi ai colori di un singolo arco del grafo. Se P^* non può impegnarsi a una colorazione valida, questo vuol dire che devono esserci almeno due nodi che pur essendo collegati da un arco hanno la stessa colorazione. Ripetendo questi passaggi per $n \cdot |E|$, dove n è il numero di nodi del grafo e $|E|$ è la cardinalità dell'insieme degli archi, il prover può farla franca con probabilità di al più $(1 - \frac{1}{|E|})^{n \cdot |E|}$, che è trascurabile, quindi la proprietà di soundness è rispettata. Per far sì che la dimostrazione sia ZK, a ogni esecuzione una nuova colorazione viene impegnata dal prover e il verifier vede i colori legati a un singolo arco per volta.

Lo strumento chiave per costruire il simulatore per questo problema è il "rewinding". S prova a indovinare un arco e e assegna una colorazione randomica valida ai due nodi ad esso associati, mentre assegna zeri a tutti gli altri nodi come colorazione. Se l'arco e' richiesto dal verifier è proprio uguale ad e , S apre i commitment dei due nodi mostrando la colorazione, terminando così la simulazione di quell'iterazione. Altrimenti, riavvolge V^* per riportarlo allo stato precedente all'iterazione⁶, per fargli scegliere una nuova challenge fino ad avere $e' = e$. Questo procedimento andrà ripetuto per il numero di iterazioni richieste. In una dimostrazione reale non ci sono archi con etichette pari a zero (quindi non valide) e non si ha la possibilità di eseguire riavvolgimenti, ma queste differenze non sono visibili al verifier.

In generale, l'esistenza di un simulatore in grado di provare affermazioni, eventualmente anche non vere, senza possedere una witness, sembra contraddire la proprietà di soundness. Invece, va sottolineato come non sia questo il caso, in quanto il simulatore S ha potere aggiuntivo che invece P non avrebbe: in questo esempio, questo potere è dato dalla capacità di riavvolgimento che può esercitare su V^* . Inoltre, la proprietà di hiding dei commitments è indispensabile per far sì che la simulazione possa concludersi con esito positivo. Ad esempio, in presenza di un verifier che può scoprire gli impegni venendo a conoscenza degli unici due nodi con colorazione valida, questo potrebbe volontariamente far fallire la simulazione proponendo sempre challenge che coinvolgono nodi con colorazioni errate o non valide, portando a un numero troppo elevato di riavvolgimenti.

2.2.2 Zero Knowledge a numero di round costante

Spesso, quando si usano schemi di questo tipo in applicazioni reali, si cerca di minimizzare il numero di round per ragioni di efficienza e praticità. Un numero non prevedibile di interazioni potrebbe infatti non essere desiderabile in contesti in cui è importante minimizzare la complessità computazionale e la latenza delle interazioni, come nei protocolli di autenticazione, e per questo negli usi pratici si tende a limitarlo fissandolo a una cifra costante relativamente piccola, pur dipendendo dai specifici requisiti di sicurezza.

Tuttavia, fissare un numero di round introduce difficoltà durante la simulazione. Considerando il problema della tricolorazione dei grafi, decidere di compattare i round svolgendo le esecuzioni in maniera parallela fa fallire le simulazioni e porta quindi a non riuscire a completare la prova che si sta ancora operando con un protocollo a conoscenza zero. Questo avviene perchè si dovrebbero indovinare contemporaneamente tutti i nodi richiesti dal verifier prima che questo li invii, perchè anche in caso di rewinding V^* potrebbe semplicemente scegliere altri nodi in maniera casuale, per cui la proprietà di riuscita è esponenzialmente ridotta. Goldreich e Kahan hanno proposto un protocollo alternativo che prevede l'invio dei commitments da parte del

⁶E' utile in questo caso dotare le due parti dei trascritti dell'interazione, così da poter agire direttamente su quello di V^* per riportarlo allo stato precedente.

verifier prima dell'inizio dell'interazione con il prover e prima che quest'ultimo invii i suoi impegni, impedendone la modifica durante il riavvolgimento [GK96].

Algorithm 2 Descrizione del protocollo di Goldreich-Kahan

- 1: Peggy: invia il primo messaggio per avviare l'interazione.
 - 2: Victor: sceglie N definito come $n \cdot |E|$ archi $e_1, \dots, e_N$, costruisce $q = e_1, \dots, e_N$ e si impegna a questa stringa usando uno schema di commitment che è perfectly-hiding.
 - 3: Peggy: impegna N colorazioni indipendenti casuali del grafo da usare nelle N esecuzioni parallele e invia il commitment per tutte utilizzando lo schema di commitment perfectly-binding.
 - 4: Victor: esegue il decommitment su q .
 - 5: Peggy: se il decommitment di q è valido, invia i decommitment corrispondenti per ogni esecuzione, altrimenti interrompe l'interazione. In particolare, se e_i è l'arco nell' i -esima esecuzione, Peggy apre il commitment sui nodi di quell'arco nell' i -esimo insieme di impegni per le colorazioni.
 - 6: Victor: solo se tutti i controlli vengono superati, accetta la prova.
-

Nella simulazione, una volta che V^* ha fissato il suo commitment, possiamo inviare degli impegni-spazzatura per far aprire quelli del verifier. Una volta ottenuti è possibile riavvolgere V^* al momento immediatamente successivo al suo primo invio per potergli consegnare stavolta delle colorazioni valide, essendo a conoscenza degli archi richiesti.

Dato che il commitment di q è perfettamente nascosto, il che significa che è computazionalmente impossibile dedurre il suo valore, questo fa sì che possa essere solo computazionalmente vincolante, rendendo possibile per V^* aprire il commitment rivelando un $q' \neq q$ e questo farebbe fallire la simulazione. Tuttavia, se questo succedesse con probabilità non trascurabile, vorrebbe dire che V^* potrebbe essere utilizzato per violare anche il vincolo computazionale dei commitments e questo non è possibile.

2.2.3 Honest-Verifier Zero-Knowledge

Un sistema di dimostrazione è HVZK se la proprietà di zero-knowledge è rispettata quando si considerano provers potenzialmente malevoli⁷, ma verificatori semi-onesti. Mentre l'obiettivo del verificatore malevolo è apertamente quello di violare la sicurezza del sistema non attenendosi al protocollo, un Verifier semi-onesto, anche detto "onesto, ma curioso", segue il protocollo, ma potrebbe cercare di ottenere informazioni aggiuntive analizzando la comunicazione. La simulazione in questo caso è molto diversa, rispetto a quella vista fin ora per far fronte a situazioni più generali con avversari malevoli, ma c'è da sottolineare che l'HVZK è una particolarizzazione e come tale potrebbe non essere sempre sufficiente nei contesti pratici, in quanto basata su assunzioni più deboli rispetto alla controparte generale.

Considerando ancora una volta il problema della tricolorazione di un grafo, in questo caso la simulazione dovrà svolgersi secondo il seguente schema:

⁷La proprietà deve essere sempre valida tenendo questo aspetto in considerazione, altrimenti non avrebbe senso costruire un sistema di dimostrazioni.

Algorithm 3 Simulatore nel caso di HVZK

```

1: Dato un grafo  $G$  e un input ausiliario  $z$ :
2: Sia  $N = n \cdot |E|$ 
3: for  $i$  da 1 a  $N$  do
4:    $S$  sceglie casualmente un arco  $e_i$  del grafo
5: end for
6:  $S$  crea  $q = (e_1, \dots, e_N)$  attraverso i lanci di monete  $r_q$ 
7: for  $i$  da 1 a  $N$  do
8:   Sia  $e_i = (v_j, v_k)$ 
9:    $S$  sceglie una colorazione valida per  $i$  due nodi e assegna agli altri l'etichetta 0
10:  Genera il vettore dei commitment  $c_i$  per l'esecuzione  $i$ , ovvero produce i commitments per tutti i nodi dei grafi come visto in precedenza, compresi quelli con colorazione non valida
11:   $S$  genera il vettore di decommitment  $d_i$  per  $i$  due nodi con colorazione valida di ogni esecuzione
12: end for
13:  $S$  genera in output la vista di un verificatore semi-onesto definito da  $\langle G, z, r_q; (c_1, \dots, c_N), (d_1, \dots, d_N) \rangle$ 

```

Dato che un verificatore onesto prenderebbe le decisioni casuali attraverso i lanci di monete, che è equivalente al leggerli dal suo nastro casuale, simulando questi lanci, S riesce a generare la vista del verifier senza bisogno di ricorrere al rewinding, ma semplicemente scegliendo casualmente le challenge. Dato che V si attiene al protocollo, S sa come si comporta e quindi di fatto può eliminare completamente l'interazione.

Questo protocollo non è black-box zero knowledge e in generale HVZK è realizzabile e dimostrabile in maniera molto più agevole rispetto a questo.

2.2.4 Non-Interactive Zero-Knowledge

E' stato presentato anche un modello non interattivo della conoscenza zero [BSMP91], che affronta il problema della ZK interattiva di dover assicurare che ci sia sempre un Verifier online e disponibile per avviare e portare avanti un'interazione potenzialmente anche lunga con il Prover, selezionando in ogni caso almeno una challenge. Grazie a una fase preliminare in cui avviene lo scambio di una Common Reference String (CRS) randomica che assume il ruolo di chiave, oppure utilizzando un modello a oracolo randomico, si riescono a eliminare sia le altre fasi di interazione che il bisogno di una challenge inviata dal Verifier. In questo modo, il Prover può generare una prova compatta che può essere verificata dal verifier, anche eventualmente in un secondo momento, in un'unica fase senza ulteriori scambi di messaggi. Quando la dimostrazione è particolarmente piccola, di dimensione costante, e facilmente verificabile (quando è quindi al più di complessità sub-lineare), si parla di ZK-SNARK (Zero-Knowledge Succinct Non-Interactive Argument of Knowledge). Un NIZK è NIZK Proof of Knowledge⁸ se soddisfa le seguenti proprietà:

⁸Una ZKP prova l'esistenza di una witness per un certo statement, mentre una ZKPoK si estende oltre, provando che non solo quell'informazione esiste, ma anche che il Prover ne è in possesso. Questo concetto è formalizzato affermando l'esistenza di un estrattore efficiente che può ottenere la testimonianza dal Prover.

1. Completezza Perfetta (Perfect Completeness): per ogni coppia di statement x e relativa witness w che prova la sua appartenenza a un certo linguaggio, il Verifier accetta con probabilità 1 una dimostrazione del prover in possesso della stessa. Se il Prover possiede la conoscenza, convince sicuramente anche il Verifier.
2. Unbounded Zero-Knowledge: esiste un simulatore Z che può generare dimostrazioni virtuali indistinguibili dalle vere dimostrazioni. Quindi, per qualsiasi tentativo di distinguere tra dimostrazioni reali e virtuali fatto da un Distinguisher D in PPT (Probabilistic Polynomial Time), la probabilità di successo è trascurabile.
3. Knowledge Soundness: è una forma più forte di soundness e implica che, oltre al fatto che il Prover non possa convincere il Verifier di una proposizione falsa, esiste un estrattore⁹ K che può estrarre la conoscenza segreta w dal prover, se questa esiste. L'output dell'estrattore è statisticamente vicino alla distribuzione uniforme. Inoltre, la probabilità che una dimostrazione falsa π superi l'estrazione è al più una quantità trascurabile rispetto alla probabilità che una vera dimostrazione superi il processo di estrazione. In pratica, un sistema è knowledge sound se, quando un P fornisce una dimostrazione, non solo la proposizione è vera, ma l'estrattore può anche estrarre la conoscenza segreta usata da P durante la dimostrazione.

Si noti infine, come la Knowledge Soundness sia una proprietà del verifier, il cui comportamento protegge da prover non onesti garantendo che conoscano la witness. Al contrario, la Zero Knowledge è una proprietà che protegge da Verifier malevoli ed è attribuibile al Prover, che con la sua condotta garantisce di non rivelare le informazioni segrete attraverso i suoi messaggi.

2.2.5 Sigma-Protocols

Gli ultimi protocolli presentati all'interno del contesto della ZK sono i Σ -Protocols, che rappresentano una caratterizzazione specifica ampiamente diffusa nelle dimostrazioni a conoscenza zero, risultando spesso più pratici nelle applicazioni pratiche. Nonostante la sottile linea che talvolta li distingue in una categoria a parte, è osservabile come essi rispettino generalmente alcune proprietà strutturali chiave, rendendo così gli ZKP un concetto più ampio e generale rispetto a questi:

- sono interattivi;
- non sono completamente ZK, ma solo HVZK;
- non sono laconici (o succinti).

Sono costituiti da tre fasi principali:

1. Fase di inizializzazione (Setup): il prover e il verifier stabiliscono i parametri del protocollo, generano le eventuali chiavi crittografiche e creano gli impegni iniziali.

⁹E' un meccanismo teorico il cui uso contribuisce a garantire la robustezza e la sicurezza dei protocolli crittografici. Il Knowledge Extractor è parte integrante di dimostrazioni di sicurezza nella progettazione di algoritmi crittografici avanzati. In pratica, può essere utilizzato per dimostrare che se un protocollo a conoscenza zero è violato, la conoscenza segreta può essere comunque estratta senza rivelare ulteriori dettagli e senza compromettere la sicurezza del sistema.

2. Fase di interazione (Interaction): ha luogo l'effettivo scambio di messaggi. Il mittente dimostra di conoscere l'informazione senza rivelarla, e V verifica questa conoscenza.
3. Fase di conclusione (Conclusion): il protocollo si conclude con l'accettazione o il rifiuto della prova di conoscenza fornita da P .

Inoltre, l'interazione si articola come segue con il susseguirsi di esattamente tre round (con lo scambio, quindi, di tre messaggi):

Algorithm 4 Schema generico della fase di interazione

- 1: Peggy: invia una richiesta di interazione a Victor, dichiarando di possedere una determinata conoscenza che è in grado di dimostrare.
 - 2: Victor: genera una sfida casuale da inoltrare a Peggy.
 - 3: Peggy: esegue i calcoli richiesti per produrre una risposta che dimostra la conoscenza senza rivelarne il valore effettivo.
-

I protocolli sigma, come menzionato all'inizio, trovano ampio impiego in svariati contesti applicativi, quali l'autenticazione, la firma digitale, l'accesso controllato e, in generale, nelle situazioni in cui è necessario dimostrare l'identità senza condividere dettagli sensibili come password o chiavi segrete tramite un canale che potrebbe essere potenzialmente non sicuro.

Come nota finale, è importante ribadire che tutti i protocolli Sigma sono protocolli ZK, ma non è altrettanto vero il contrario.

2.3 Predictable Arguments of Knowledge

E' di recente proposta il concetto di Predictable Arguments of Knowledge [FNV17], in cui si introduce la nozione di prevedibilità nell'ambito dei protocolli interattivi. La caratteristica principale è che la risposta del Prover può essere predetta dati i risultati dei lanci delle monete private del verifier, che ne determinano la strategia.

Un Predictable Argument of Knowledge (PAoK) è un protocollo interattivo multi-round (P, V) in cui, per ogni round, il verifier V genera, attraverso i lanci delle proprie monete private r , una challenge su un input comune x da inviare al prover P , unitamente alla risposta prevista b . P , dato l'input x , la relativa witness w e la challenge c generata da V elabora una risposta a . Se per tutti i round risulta $a = b$, P convince con successo V . In questo caso, un singolo "round" comprende un messaggio dal verifier al prover e quello di risposta in direzione opposta. Se esiste un prover che convince il verifier con una certa probabilità Pr , la proprietà di extractable security fa sì che sia possibile estrarre una witness con probabilità strettamente legata a Pr .

Grazie alla possibilità di sapere in anticipo le risposte, è possibile generare le challenge in maniera simultanea, per poi inviarle al prover in modo invece sequenziale, una per round.

Supponendo di avere un PAoK con un certo numero ρ di round. La coppia di algoritmi (**Chall**, **Resp**) descrivono un PAoK con il seguente schema generale:

Algorithm 5 Generico protocollo PAoK

-
- 1: V : genera randomicamente $(\mathbf{c}, \mathbf{b})$ attraverso l'algoritmo $Chall(1^\kappa, x)$, dove $\mathbf{c}$ è definito come la ennupla delle challenge per ogni singolo round e $\mathbf{b}$ come la ennupla delle rispettive risposte attese, ovvero $\mathbf{c} := (c_1, \dots, c_\rho)$, $\mathbf{b} := (b_1, \dots, b_\rho)$.
 - 2: **for** i da 1 a ρ **do**
 - 3: V invia la challenge c_i a P
 - 4: P calcola $(a_1, \dots, a_i)$ con l'algoritmo **Resp** $(1^\kappa, x, w, c_1, \dots, c_i)$ e invia la risposta a_i a V
 - 5: V controlla che $a_i = b_i$: se corretto si va avanti con il prossimo round, altrimenti ritorna **rej**
 - 6: **end for**
 - 7: Se tutte le challenge risultano superate, V ritorna **acc**
-

Il κ utilizzato come input per i due algoritmi è il parametro di sicurezza considerato tale che $\kappa \in \mathbb{N}$.

Si noti come l'algoritmo **Resp** prenda in input anche tutte le challenge passate fino a quella del round i corrente per generare l' i -esima risposta. Nello schema mostrato, l'algoritmo di risposta in realtà genera nuovamente a ogni round anche tutte le risposte precedenti, ma questo può anche essere evitato, facendo sì che restituisca solamente la nuova risposta. Anche i PAoK devono rispettare le seguenti due proprietà di completezza e correttezza:

- Completeness: per tutte le coppie (x_κ, w_κ) appartenenti alla relazione R^{10} il sistema ha una probabilità molto alta¹¹ di accettare la dimostrazione del prover che lavora in PPT.
- Knowledge soundness: se il prover P ha una buona probabilità di convincere il verifier V , allora l'estrattore di conoscenza K ha una buona probabilità di estrarre un'informazione y tale che $f(w) = y$ per una specifica funzione f e $(x, w) \in R$.

E' di interesse notare che è stato dimostrato che l'esecuzione in ripetizioni parallele, generalmente, non amplifica la correttezza dei protocolli Arguments of Knowledge [BIN97, PW12], ma lo fa nel caso particolare dei PAoK con singolo round, in quanto anche secret-coin.

2.3.1 PAoK laconici

E' sempre possibile convertire un normale PAoK con più round e risposta di un generico numero di bit l per farlo diventare estremamente laconico sia in termini di numero di round che in termini del numero l di bit per la risposta del prover. Questo vuol dire che il PAoK risultante è costituito da un singolo round di interazione (quindi da due messaggi) con una risposta dal prover di un singolo bit ($l = 1$). Per fare questo, partendo da un generico (P, V) , se ne definisce uno nuovo (P', V') che si comporta come segue:

- V' esegue V per ottenere una coppia (c, b) come definiti in precedenza, genera casualmente r e calcola la nuova risposta attesa dal prodotto interno tra b e r ¹².

¹⁰ R è la relazione associata a un linguaggio L , ovvero: $x \in L \iff \exists w$ tale che $(x, w) \in R$.

¹¹Di $1 - \nu(\kappa)$, che è una funzione trascurabile nel parametro di sicurezza che rappresenta l'errore da considerare nelle macchine probabilistiche.

¹²Il prodotto interno è un'operazione che dati due vettori restituisce uno scalare.

- P' esegue P per ottenere la risposta a , avendo come input la challenge (c, r) , e calcola la risposta da inviare a V' tramite il prodotto interno di a e r .
- Infine, esattamente come prima, V' confronta a e b .

Di seguito vengono presentate alcune osservazioni sui recenti sviluppi dei PAoK, con diversi tipi degli stessi contraddistinti da caratteristiche diverse.

2.3.2 PAoK e Extractable Witness Encryption

Il concetto di PAoK è fortemente legato a quello di Extractable Witness Encryption. Uno schema di Witness Encryption (WE) permette di cifrare un messaggio utilizzando uno statement $x \in L$, con L che è un linguaggio in NP, in modo tale che tutti e soli i possessori di una witness w per x possano decifrare il messaggio. Avendo uno schema WE con una coppia di algoritmi ($Encrypt, Decrypt$) che lavorano in PPT, possiamo dire che sia una Extractable Witness Encryption (Ext-WE) se lo schema rispetta anche la proprietà di Extractable Security, ovvero: da qualsiasi avversario che violi la sicurezza semantica¹³ del cifrario, possiamo ottenere un extractor che calcola una testimonianza w valida per x senza dover decifrare direttamente il messaggio.

Data una NP-relation R , esiste un PAoK per R se e solo se esiste uno schema di Ext-WE per la stessa relazione, in quanto i due schemi risultano essere equivalenti. Per rendere evidenti le implicazioni in entrambi i versi, si noti che:

- Per passare da Ext-WE a PAoK possiamo cifrare un bit randomico con lo schema di cifratura e poi chiedere a P di ritornare il bit corretto.
- Per passare da PAoK a Ext-WE, prima si costruisce lo schema PAoK estremamente laconico partendo da quello dato, si genera una coppia di challenge e risposta (c, a) e infine si cifra un singolo bit β come $(c, a \oplus \beta)$ (che quindi andrà a costituire il ciphertext).

2.3.3 PAoK e Zero-Knowledge

Dato che le risposte per le challenge correttamente generate da un verifier possono essere predette senza essere a conoscenza di una witness valida (non c'è infatti nessun vincolo che imponga a V di esserne a conoscenza), è possibile affermare che i Predictable Arguments of Knowledge sono sempre Honest-Verifier Zero-Knowledge, ma trasformarne uno in un altro equivalente che soddisfi la proprietà di zero-knowledge senza il vincolo di avere un verificatore semi-onesto non è altrettanto facile. Di seguito viene illustrata una delle soluzioni proposte in [FNV17].

Common Reference String Model

In questo modello, i protocolli interattivi vengono eseguiti come descritti all'inizio di questo capitolo, ma in più si aggiunge una fase iniziale in cui viene generata una stringa binaria casuale ω che viene poi condivisa da entrambe le parti del protocollo, che possono utilizzarla in vari modi, per esempio per generare chiavi segrete, per creare gli impegni crittografici o come parametro nel processo di dimostrazione.

¹³Si tratta di una proprietà che richiede che il cifrario nasconda completamente l'informazione contenuta nel messaggio, in modo tale che l'attaccante non sia in grado di ottenere alcuna informazione utile dal messaggio cifrato, anche se già a conoscenza di altri messaggi cifrati che corrispondono a messaggi diversi.

Partendo da un generico PAoK, se ne costruisce uno che opera in CRS. Perchè si riesca a raggiungere la proprietà di ZK, occorre che il verifier generi insieme alla sfida, utilizzando la stringa ω , una prova di conoscenza NIZK-PoK che la sfida è ben formata secondo il PAoK originale, che invia a P insieme alla challenge. In questo modo, il prover può utilizzare ω per verificare la correttezza della sfida e, in caso lo sia, rispondere come farebbe nel PAoK normale (e altrimenti interrompere la comunicazione).

2.3.4 Predictable ZAPs

Un protocollo ZAP è un protocollo public-coin che mira a generare la challenge del verifier in maniera indipendente dall'istanza x considerata. Un Predictable ZAP (PZAP) vuole mantenere questo aspetto aggiungendo il concetto di prevedibilità, in modo da poter usare la stessa sfida in più esecuzioni del PAoK. La risposta può essere predetta attraverso le monete private di V e una funzione botola (trapdoor function). Al contrario delle ZAP, questi nuovi schemi sono private-coin e inoltre non riescono a raggiungere la proprietà di indistinguibilità per la witness¹⁴.

2.3.5 Commit-and-Prove Predictable Arguments

Come ultimi sviluppi trattati in questo capitolo, è di interesse citare i CPPA. [Kho22] propone una forma diversa e più rilassata di prevedibilità, suddivisa in una fase detta di commitment e una di challenge/response. Inizialmente il prover impegna una witness e invia il commitment al verificatore, per poi seguire normalmente un'interazione PAoK tra le due parti, con la particolarità che le sfide del verifier possono dipendere dal commitment ricevuto. Questo approccio consente a V di fare previsioni basate sull'impegno effettuato dalla controparte, ma senza la necessità di prevedere completamente le risposte specifiche, in quanto si ha a disposizione l'informazione aggiuntiva dell'impegno generato precedentemente. Un CPPA è uno schema multi-round che si compone di tre algoritmi (**Commit**, **Chall**, **Resp**). Dal primo di questi dipende la fase iniziale.

I CPPA, come i PAoK, possono anche essere resi estremamente laconici in termini del numero dei round e della complessità della dimostrazione. La valenza pratica di questo modello viene incrementata, inoltre, dalla possibilità di essere integrato con la proprietà di conoscenza zero (zero knowledge) nel modello NPRO (Non-Programmable Random Oracle model).

¹⁴Witness Indistinguishability, WI

Capitolo 3

Panoramica su SCAPI

SCAPI¹⁵ [9, 10] è una raccolta di librerie open-source sviluppata dal Cryptography Research Group della Bar-Ilan University.

Pur esistendo alcune librerie crittografiche general-purpose, la maggior parte delle stesse sono sviluppate su misura per affrontare problemi specifici di cifratura e autenticazione, incentrandosi sull'area della comunicazione sicura, mentre molto più raramente si occupano della computazione sicura. Inoltre, spesso non viene utilizzata la terminologia considerata nello studio di protocolli crittografici e viene forzato l'uso di specifiche implementazioni, che possono rivelarsi problematiche quando si va avanti nello sviluppo. Al contrario, SCAPI è una libreria¹⁶ general-purpose focalizzata sulla secure computation. Scritta in Java, ne conserva le qualità, quali:

- portabilità multi-piattaforma
- flessibilità
- versatilità
- scalabilità
- modularità
- possibilità di riutilizzo del codice
- praticità di implementazione rispetto ai linguaggi a basso livello come C

e in più, grazie al JNI¹⁷ framework, permette al codice Java di interagire con codice scritto in altri linguaggi, in particolare andando più a basso livello con codice nativo oppure C/C++, ed ereditarne quindi l'efficienza. Anche grazie a quest'ultima caratteristica, SCAPI contiene e si serve di librerie esistenti quali BouncyCastle [1] e OpenSSL [7].

Essendo un'infrastruttura generale basata su gerarchie di interfacce per i diversi livelli di sicurezza che si potrebbero voler raggiungere, nuove implementazioni possono essere aggiunte all'evenienza, in quanto basterà solo far implementare a queste ultime, l'interfaccia corrispondente alla primitiva che si vuole modificare. Nel caso

¹⁵Secure Computation Application Programming Interface o Secure Computation API.

¹⁶Definita "infrastruttura" dal Professor Yehuda Lindell [8], termine con cui si indica che tale libreria fornisce una base o un supporto fondamentale per lo sviluppo di applicazioni o servizi più articolati, gestendo gli aspetti trasversali e mantenendo al contempo l'astrazione dei dettagli più complessi.

¹⁷Java Native Interface.

del rilascio di una nuova libreria, ad esempio, basterà scrivere un wrapper SCAPI per incapsulare la libreria stessa e tradurne le funzionalità nel formato e nell'interfaccia definiti da SCAPI, consentendone così un utilizzo coerente all'interno dei protocolli già esistenti.

3.1 Architettura di SCAPI

La struttura generale di SCAPI è divisa in tre livelli o strati:

- primitive di base
- schemi non interattivi
- protocolli interattivi

Nell'immagine sottostante [2] si può osservare più nel dettaglio l'organizzazione in layers.

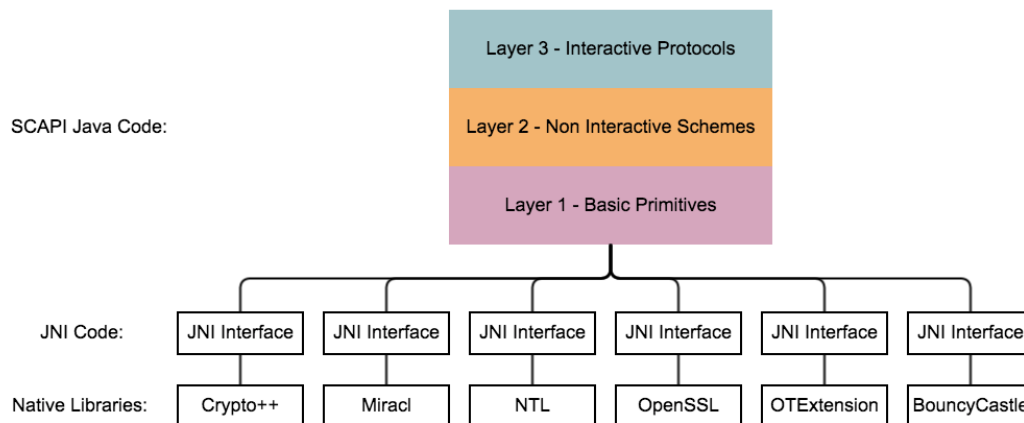


Figura 3.1. Rappresentazione grafica dei layers

3.1.1 Communication Layer

Oltre a quelli rappresentati, è presente un ulteriore livello che include interfacce di comunicazione a due parti o multiparte che è possibile utilizzare qualora le dimensioni o la natura del progetto lo richiedano, in caso di implementazione, ad esempio, di protocolli distribuiti complessi in cui si punta a un alto grado di interattività tra le parti coinvolte. La tipologia più semplice di comunicazione viene raggiunta attraverso un canale non autenticato e non cifrato, mentre canali sicuri possono essere ottenuti utilizzando il protocollo TLS¹⁸ o mediante l'uso di una chiave simmetrica precondivisa. Anche più canali possono essere stabiliti tra una singola coppia di partecipanti e questa caratteristica risulta estremamente importante per i protocolli in cui è richiesto il multithreading per la gestione di esecuzioni parallele. L'utente ha la possibilità di scegliere tra tre tipi di canali differenti:

- canale basato su socket;

¹⁸Transport Layer Security.

- canale basato su coda, in generale più robusto¹⁹, ma meno efficiente, di un canale basato su socket;
- canale TLS/SSL (disponibile per entrambe le opzioni socket e coda), per fornire una comunicazione sicura, che può risultare a seconda dei casi più conveniente rispetto all'uso di chiavi precondivise.

3.1.2 Layer 1: Basic Primitives

Tra le primitive che è possibile trovare ci sono funzioni di hash crittografico, Pseudorandom Functions (PRF), Pseudorandom Permutations (PRP) e Trapdoor Permutations. E' bene ricordare che la sicurezza o meno di una primitiva crittografica molto spesso dipende dal contesto in cui la si considera, in base ai tipi di avversari e di attacchi che si vogliono contrastare. Ad esempio, uno schema che è sicuro in presenza di EAV²⁰ potrebbe non esserlo nel caso dei chosen-plaintext attacks (CPA), ma qualsiasi schema che è sicuro in presenza di chosen-ciphertext attacks (CCA) lo è anche in presenza di CPA e di EAV. Con questo esempio si può notare che questi livelli di sicurezza formano in questo caso una gerarchia, concetto riproposto e utilizzato in SCAPI [3].

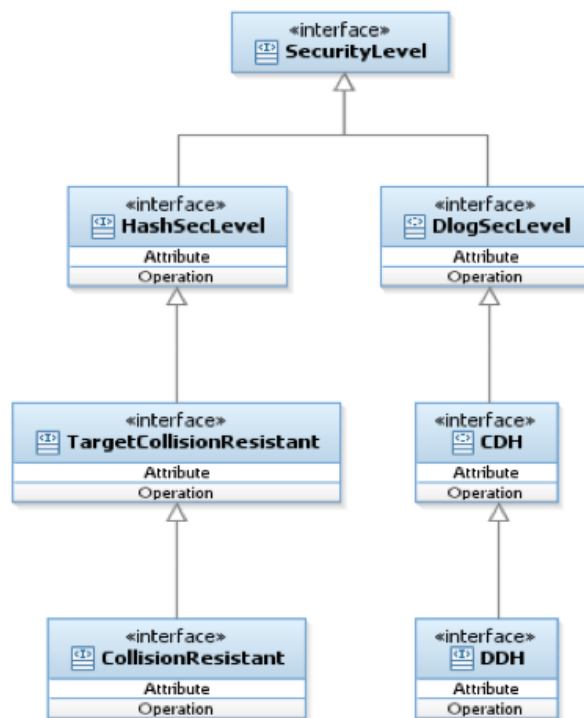


Figura 3.2. Esempio di gerarchia di livelli di sicurezza in SCAPI

Infine, si sottolinea come la scelta del livello di sicurezza per una particolare applicazione pratica dovrebbe basarsi su attente analisi per bilanciare gli obiettivi

¹⁹Riprendendosi automaticamente da problemi o fallimenti di comunicazione.

²⁰Eavesdropping adversaries.

di sicurezza ed efficienza richiesti dal caso specifico, in quanto i livelli potrebbero essere non comparabili oppure scegliere semplicemente lo schema più sicuro potrebbe portare a gravi inefficienze.

3.1.3 Layer 2: Non-Interactive Schemes

Questo strato si basa sulle primitive del precedente per la computazione delle varie operazioni richieste per implementare diversi schemi di:

- crittografia simmetrica;
- crittografia asimmetrica;
- autenticazione dei messaggi;
- firme digitali.

3.1.4 Layer 3: Interactive Protocols

I protocolli interattivi contenuti in questo layer possono basarsi sulle primitive di base. In più, possono essere implementati per essere usati in modo indipendente oppure per essere combinati tra loro per realizzare schemi più raffinati. Questi protocolli coinvolgono tipicamente due partecipanti che assumono ruoli diversi e definiti. La comunicazione tra queste due parti può essere realizzata, in contesti avanzati, attraverso gli strumenti messi a disposizione dal Communication Layer. Sono inclusi Sigma Protocols, protocolli OT, ZKP e ZK-PoK, schemi di commitment e di coin tossing, e in più sono già fornite anche alcune combinazioni degli stessi, come ad esempio le classi *ZKFromSigmaProver* e *ZKFromSigmaVerifier*.

3.2 Utilizzo nel progetto di tirocinio

Nella realizzazione pratica della DDH assumption si è scelto di usufruire delle interfacce astratte ZK presenti all'interno del terzo layer appena presentato, in particolare:

- ZKProver, che simula il comportamento di un prover fornendo la segnatura del metodo `prove()`, che accetta un input contenente i dati necessari per computare la prova.

```

1 public abstract interface ZKProver{
2     public abstract void prove(ZKProverInput arg0);
3 }

```

- ZKProverInput, interfaccia astratta e vuota, che funge da marcatore e deve essere implementata concretamente dalle classi che si vuole usare come input del metodo `prove()` dello ZKProver.
- ZKVerifier, che invece ha il compito di simulare il comportamento del verifier e mette a disposizione la segnatura del metodo anch'esso astratto `verify()`, che gestisce in input i dati necessari per portare avanti il processo di verifica e restituisce un valore booleano in output, *true* se la prova del Prover risulta corretta e verificata, *false* altrimenti.

```
1 public abstract interface ZKVerifier{  
2     public abstract boolean verify(ZKCommonInput arg0);  
3 }
```

- ZKCommonInput, interfaccia astratta e vuota con funzione di marcatore. Viene usata esplicitamente per uniformare l'input del metodo verify() del Verifier che implementa ZKVerifier, ma può essere utilizzata anche in altri contesti e per altri tipi di messaggi da entrambe le parti che partecipano al protocollo.

Capitolo 4

Implementazione di un protocollo Zero-Knowledge

4.1 Decisional Diffie-Hellman assumption

Il problema considerato per l'implementazione è l'assunzione, o ipotesi, decisionale Diffie-Hellman, abbreviato in DDH, che si basa principalmente sull'utilizzo dei gruppi ciclici e sul problema del logaritmo discreto.

Gruppi Ciclici

Un gruppo, in matematica, è una struttura algebrica che consiste in un insieme non vuoto su cui è definita un'operazione binaria che rispetta l'assioma di associatività, per cui esiste un elemento neutro e per cui ogni elemento ha un proprio inverso. Dato un gruppo G , se esiste un elemento g , detto generatore, in grado di generare tutti gli elementi del gruppo attraverso l'operazione definita, allora G si definisce gruppo ciclico. In caso di gruppo ciclico moltiplicativo, ovvero quello di interesse per il problema affrontato, $G = \langle g \rangle = \{g^n \text{ t.c. } n \in \mathbb{Z}\}$

Nel caso di un gruppo ciclico finito, l'ordine del generatore è il minimo intero n per il quale g^n è pari all'elemento neutro, 1 del caso in cui l'operazione definita sia la moltiplicazione.

Se G ha un ordine pari a p e p è un numero primo, allora G è ciclico e ogni elemento diverso dall'elemento neutro è un suo generatore²¹.

Problema del logaritmo discreto

Nell'ambito dell'aritmetica modulare, i logaritmi discreti corrispondono ai logaritmi ordinari. L'operazione inversa di questi logaritmi, invece di essere l'esponenziale, è la potenza discreta. Avendo un gruppo ciclico G generato da un elemento g , il problema del logaritmo discreto può essere visto come:

"Dati g^a e g in G , si trovi l'intero a "

La difficoltà nel risolvere questo tipo di problemi, rispetto ai logaritmi ordinari, è legata alla struttura del gruppo in cui viene definito. Il problema si può riformulare chiedendo di trovare l'esponente a per cui $g^a \equiv x \pmod{p}$, per g^a e g dati in G . Per g e p sufficientemente grandi, non è noto un algoritmo che risolva questa congruenza modulare in maniera efficiente, quindi in tempo polinomiale, e per questo motivo

²¹In generale questo è dovuto al fatto che nei gruppi ciclici ogni generatore è sempre coprimo con l'ordine del gruppo. Possono esserci, quindi, più generatori per lo stesso gruppo.

sono alla base di un notevole numero di sistemi e schemi crittografici, che basano su di essi la propria sicurezza.

Algoritmo DDH

Nel contesto della crittografia a chiave pubblica, il problema di Diffie-Hellman coinvolge l'idea di scambiare segreti attraverso canali di comunicazione non sicuri. Sia G un gruppo ciclico di ordine primo, g un generatore di G , e p l'ordine di G . Il problema Diffie-Hellman può essere formulato come: dato g^a e g^b , trovare g^{ab} . Invece, la Decisional Diffie-Hellman assumption afferma che è computazionalmente difficile distinguere, senza conoscere l'esponente segreto a o b e dati solamente g , g^a , e g^b , tra le due coppie:

$$\begin{array}{l} g, \quad g^a, \quad g^b, \quad g^{ab} \\ g, \quad g^a, \quad g^b, \quad g^c \quad (\text{dove } c \text{ è scelto casualmente da } \{1, 2, \dots, p-1\}) \end{array}$$

L'implementazione è basata sulla Chaum-Pedersen ZKP applicata alla Decisional Diffie-Hellman assumption, in cui il verifier desidera verificare che il prover possieda un esponente segreto che renda la DDH difficile da violare. La prova è strutturata seguendo l'algoritmo qui presentato:

Algorithm 6 Chaum-Pedersen ZKP per DDH

- 1: Siano p un valore primo e g un generatore di un gruppo ciclico, entrambi abbastanza grandi e condivisi da entrambi i partecipanti;
 - 2: Peggy è in possesso di un valore segreto a e genera la chiave pubblica $A = g^a$;
 - 3: Victor è in possesso di un valore segreto b e genera la chiave pubblica $B = g^b$;
 - 4: Peggy e Victor si scambiano le chiavi pubbliche A e B e calcolano C , il valore di una chiave segreta condivisa:
Peggy: $C = (g^b)^a \bmod p = B^a \bmod p$;
Victor: $C = (g^a)^b \bmod p = A^b \bmod p$;
 - 5: Victor invia una challenge s a Peggy, con s estratta randomicamente da $\mathbb{Z}_p$;
 - 6: Peggy genera un valore casuale r , estratto da $\mathbb{Z}_p$;
 - 7: Successivamente, Peggy inoltra a Victor tre valori:
 $y_1 = g^r \bmod p$;
 $y_2 = B^r \bmod p$;
 $z = r + a \cdot s \bmod (p-1)$
 - 8: Victor esegue due controlli e, se entrambi verificati, accetta:
 $g^z = A^s \cdot y_1 \bmod p$;
 $B^z = C^s \cdot y_2 \bmod p$;
-

Alla fine, se si ha responso positivo, il verifier è convinto che il prover sia a conoscenza del segreto a , poichè:

$$\begin{aligned} A^s \cdot y_1 &= (g^a)^s \cdot g^r = g^{a \cdot s} \cdot g^r = g^{a \cdot s + r} = g^z \pmod{p} \\ C^s \cdot y_2 &= (g^{a \cdot b})^s \cdot B^r = g^{a \cdot b \cdot s} \cdot g^{b \cdot r} = g^{a \cdot b \cdot s + b \cdot r} = (g^b)^{a \cdot s + r} = B^z \pmod{p} \end{aligned}$$

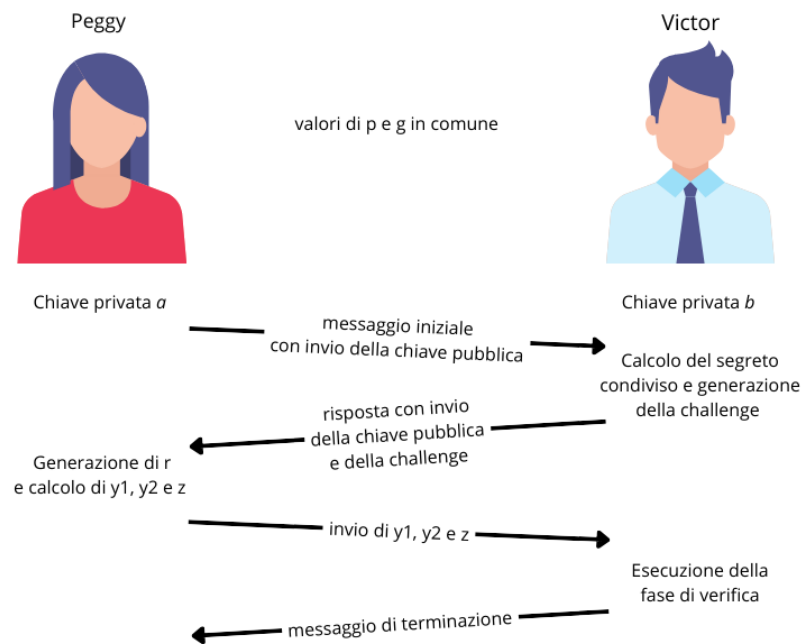


Figura 4.1. Fasi seguite nell'implementazione [4]

4.2 Scelte e prove preliminari

Nella fase precedente la messa in pratica di questo protocollo sono state valutate diverse opzioni implementative. Prime tra tutte, si sono presentate le scelte riguardo il linguaggio di programmazione e la libreria da utilizzare. Sono stati presi in considerazione principalmente i linguaggi Python e Java, entrambi molto usati in contesti crittografici. Dopo aver valutato alcune librerie Python, tra cui PyCryptodome e Cryptography, si è deciso di continuare la fase pratica del progetto di tirocinio in Java, con la libreria SCAPI.

La libreria è stata scelta per i motivi indicati nel precedente capitolo, uniti anche alla valutazione sulla libertà implementativa che SCAPI fornisce: seppur usando le interfacce apposite per la ZK, queste sono per lo più marcatori che permettono di tenere ben separate le classi corrispondenti ai due ruoli, ma non vincolano a seguire implementazioni specifiche, e questo, oltre a rendere le interfacce perfettamente adattabili in tutti i vari contesti, è stato particolarmente interessante in quanto, come primo approccio alle implementazioni crittografiche, ha permesso di rilassare i vincoli su alcuni aspetti per poter portare il focus sull'applicazione dei concetti più legati alla ZK.

Dopo alcune prove preliminari più immediate in Python, volte principalmente a verificare la comprensione e la correttezza dell'algoritmo senza concentrarsi sull'effettiva creazione e separazione dei due ruoli di prover e verifier (che infatti in questa prima fase erano ancora volutamente assenti), si è passati completamente all'approccio in Java, dopo l'importazione delle librerie necessarie e la realizzazione di piccoli esempi preliminari per testarne e capirne il funzionamento.

La preferenza per il linguaggio di programmazione Java è stata determinata dalla sua notevole somiglianza con le caratteristiche richieste per la realizzazione del sistema. La divisione in classi di oggetti e la proprietà di incapsulamento in Java si allineano in modo immediato ai ruoli di prover e verifier nei protocolli Zero-Knowledge, semplificando notevolmente la realizzazione delle astrazioni del sistema.

4.3 Implementazione

Prover e verifier non comunicano direttamente tra loro; piuttosto, la loro interazione è regolata da una classe intermedia attraverso un singolo thread, che gestisce volta per volta i messaggi scambiati inserendoli in una coda. Per quanto riguarda i valori, si è fatto un uso significativo del tipo non primitivo `BigInteger`, che supporta operazioni nell'aritmetica modulare.

Prima di raggiungere la versione finale, sono state effettuate prove testando diversi generatori di chiavi e diversi approcci per i valori in comune di p primo e del generatore g del gruppo ciclico. In particolare, si è inizialmente testato l'algoritmo con valori preimpostati e di piccole dimensioni, aggiungendo successivamente il fattore randomico solo per la creazione delle chiavi, lasciando fissi p e g . Il metodo di generazione delle chiavi private, da cui sono state poi derivate le pubbliche, ha inizialmente coinvolto l'uso di seed per generazioni in realtà pseudo-randomiche, al fine di facilitare la fase di debugging. Successivamente, si è eliminato l'uso dei seed e si è fatto ricorso alla classe **SecureRandom** di `java.security` per generare numeri crittograficamente robusti. Infine, è stata implementata la classe **GroupGenerator** che, con i metodi `generatePrime()` e `generateGenerator()`, consente di estrarre in modo randomico, attraverso appositi controlli e utilizzando nuovamente `SecureRandom`, anche i valori per g e p :

- `generatePrime(int bitLength)` genera un numero primo specificando in input la lunghezza desiderata in numero di bit. Fa uso di `probablePrime()`, un metodo probabilistico della classe `BigInteger`, per estrarlo in modo efficiente e con una sicurezza relativamente alta che sia primo. La probabilità che il `BigInteger` restituito non lo sia non supera, infatti, 2^{-100} .
- `generateGenerator(BigInteger p)` prende in input il primo precedentemente generato, che viene inteso come l'ordine di un gruppo ciclico, e ritorna in output un nuovo `BigInteger`, anch'esso primo se non con probabilità trascurabile e con la stessa lunghezza in bit di p , che rispetta i criteri per un generatore di un gruppo ciclico.

```

1 public static BigInteger generateGenerator(BigInteger p) {
2     int bitLength = p.bitLength();
3     BigInteger two = BigInteger.valueOf(2);
4     BigInteger pMinusOne = p.subtract(BigInteger.ONE);
5     BigInteger g;
6     do {
7         g = new BigInteger(bitLength, new SecureRandom());
8     } while (g.compareTo(two) < 0 || g.compareTo(pMinusOne) >= 0 ||
9             !g.modPow(pMinusOne.divide(two), p).equals(BigInteger.ONE));
10    return g;
11 }

```

Il ciclo do-while si assicura che il candidato proposto per g sia compreso tra 2 e $p-2$ estremi inclusi e che inoltre non sia un residuo quadratico modulo p . Questi controlli sono necessari poichè valori che non rispettano queste caratteristiche potrebbero introdurre, in contesti crittografici, vulnerabilità sfruttabili da un avversario malevolo.

Tutta questa prima fase è stata svolta costruendo un'interazione tra un singolo prover e un singolo verifier. Come passo finale del progetto di tirocinio, si è esteso e generalizzato il modo in cui lavorano, in particolare, il verifier e il thread di esecuzione, per prevedere comunicazioni con più prover per volta, gestendo in modo sequenziale i messaggi che arrivano nella coda dai vari prover. Ulteriori informazioni saranno fornite nei successivi paragrafi dedicati alle singole componenti.

Messaggi

Come anticipato, gli scambi sono organizzati in modo da dividere ulteriormente provers e verifier in modo che tutti i messaggi abbiano un tipo specifico scelto tramite una classe enumerativa, e i metodi dei partecipanti che rappresentano le varie fasi del protocollo prendano in input messaggi di un tipo specifico. In generale, il prover accetta esclusivamente messaggi che estendono la classe `VerifierMessage` e il verifier accetta quelli che estendono la classe `ProverMessage`. Si sottolinea ancora una volta che i messaggi sono gli unici a portare informazioni necessarie per le varie elaborazioni e che queste informazioni sono accessibili esclusivamente tramite i metodi getter di questi, quindi la comunicazione non è mai diretta. In questa fase si fa quindi molto uso dell'ereditarietà e dell'incapsulamento messi a disposizione da Java, con anche metodi privati accessibili solo all'interno delle singole classi per eseguire elaborazioni interne. Le classi `Message`, `ProverMessage` e `VerifierMessage` sono astratte e non implementabili concretamente, ma mettono a disposizione i costruttori di base per impostare il tipo del messaggio.

I campi delle classi di questa gerarchia sono final, in quanto una volta creati i messaggi, la loro unica funzione è quella di fare da tramite, senza poter in alcun modo essere modificati.

Con l'ultima versione dell'implementazione, che ha visto l'introduzione di più provers, si è resa necessaria l'associazione a ogni messaggio dell'identificatore univoco del prover a cui si riferisce o da cui è stato generato. L'unica eccezione, è per il tipo di messaggi `VerifierEndCommMessage` che, essendo mirato a chiudere del tutto il thread di comunicazione e quindi non riferendosi a nessun prover in particolare, non ne ha bisogno. Questo ha portato all'uso dell'overloading per il costruttore del `VerifierMessage` e del `Message`, in modo che in questo caso non debba essere comunicato alcun valore per quel campo:

```
1 public abstract class VerifierMessage extends Message {
2     public VerifierMessage(MessageType type) {
3         super(type);
4     }
5
6     public VerifierMessage(MessageType type, UUID proverUniqueID) {
7         super(type, proverUniqueID);
8     }
9 }
```

```

1 public abstract class Message {
2     private MessageType type;
3     private UUID proverUniqueID;
4
5     public Message(MessageType type, UUID proverUniqueID) {
6         this.type = type;
7         this.proverUniqueID = proverUniqueID;
8     }
9
10    public Message(MessageType type) {
11        this.type = type;
12    }
13 }

```

La classe ProverProofMessage implementa l'interfaccia SCAPI ZKCommonInput, in modo da poter essere l'unico tipo di messaggio da poter passare in input al metodo di verifica del verifier.

La classe VerifierChallengeMessage implementa invece l'interfaccia ZKProverInput per poter essere passata in input nel metodo prove() del prover.

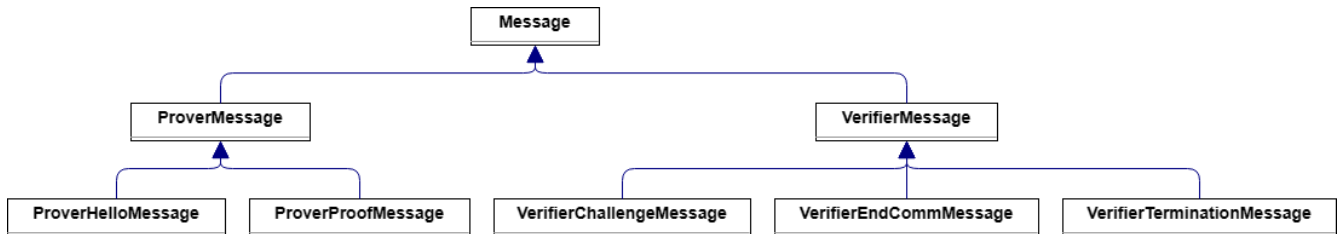


Figura 4.2. Gerarchia di messaggi [5]

4.3.1 Creazione del Thread di esecuzione

Il thread di esecuzione è creato dal metodo main(). La classe principale in cui si trova, oltre a contenere i getter per i valori di p e g , per metterli in comune nei partecipanti del protocollo, inizializza i provers e il verifier e fa partire il thread che li mette in comunicazione con le istruzioni:

```

1 Thread messageHandlerThread = new Thread(() ->
2     messageHandler(verifier, messagesQueue, proversList));
3     messageHandlerThread.start();

```

Il thread viene creato passandogli in input una lambda expression per fargli eseguire il metodo messageHandler() con i parametri specificati, ovvero:

- verifier: la singola istanza del verifier per questo protocollo,
- messagesQueue: la coda dei messaggi da gestire,
- proversList: l'ArrayList contenente tutti i provers che partecipano al protocollo.

Nell'implementazione viene usata una coda bloccante realizzata con la Linked-BlockingQueue, una struttura dati che segue il principio di "producer-consumer" (produttore-consumatore) e fornisce un modo sicuro per il trasferimento di dati. Essendo bloccante, funziona secondo questo schema generale:

- il producer inserisce dati nella coda. Se la coda è piena, il produttore viene messo in attesa finchè non ci sia nuovamente spazio disponibile;
- il consumer rimuove dati dalla coda per utilizzarli in qualche modo. Se la coda è vuota, viene messo in attesa finchè non ci siano nuovi dati disponibili;
- le operazioni sono bloccanti. Quando un produttore cerca di inserire dati mediante il metodo `put()` in una coda piena, o di prelevarli con il metodo `take()` in una coda vuota, l'operazione si blocca fino a quando non sia di nuovo possibile la sua esecuzione.

Può essere specificata in input, durante la creazione, la capacità desiderata per la coda.

Si può notare come in questo caso `provers` e `verifier` assumano in maniera alternata entrambi i ruoli.

Nel metodo `messageHandler()`, attraverso un `while(true)` (il thread viene terminato dal `verifier` mediante il messaggio adeguato) viene prelevato di volta in volta un singolo messaggio e con uno `switch-case` si decide a quale partecipante inoltrarlo, in base al tipo del messaggio stesso. Se il destinatario è un prover (che deve ricevere quindi una sfida), questo viene selezionato dalla lista con l'utilizzo degli stream di Java, filtrando il prover con l'UUID adatto, se presente (in caso non sia presente, tuttavia, ci si trova in presenza di una situazione di errore o eccezione, in quanto in condizioni normali un `verifier` manda messaggi esclusivamente a prover che sono ancora attivi e con cui ha già iniziato una conversazione):

```

1     proversList.stream()
2         .filter(prover -> prover.getUniqueID() ==
3             message.getProverUniqueID())
4         .findFirst()
        .ifPresent(prover ->
            prover.receiveChallenge((VerifierChallengeMessage)
                message));

```

Gli ultimi due metodi messi a disposizione in questa classe principale sono *`insertVerifierMessage(VerifierMessage message)`* e *`insertProverMessage(ProverMessage message)`*, che inseriscono i messaggi nella coda. Sono specifici per prover e verifier in modo da creare un'ulteriore separazione tra le parti, potendo accettare in input solo i messaggi dei tipi corrispondenti.

4.3.2 Prover

Ogni prover che vuole prendere parte al protocollo deve implementare l'interfaccia `ZKProver` di SCAPI, che fornisce la segnatura del metodo *`prove()`*. Prima di descriverlo in maniera più approfondita, si presentano brevemente di seguito anche le altre caratteristiche e gli altri metodi rilevanti. Ad ogni prover viene associato un identificativo univoco mediante la classe `UUID` (Universally Unique Identifier), generato grazie a un generatore pseudo-randomico e crittograficamente robusto di numeri. Questo identificativo permette al `verifier` di distinguere tra più prover. Dopo aver assegnato questo `UUID`, il costruttore della classe chiama il metodo `generateKeys()` per generare la chiave pubblica e quella privata mediante la classe `SecureRandom()`:

```

1     private void generateKeys() {
2         BigInteger minKey = BigInteger.valueOf(2);

```

```

3     BigInteger maxKey = P.subtract(BigInteger.valueOf(2));
4     privateKey = new BigInteger(maxKey.bitLength(), new
        SecureRandom()).mod(maxKey.subtract(minKey)).add(minKey);
5
6     publicKey = G.modPow(privateKey, P);
7 }

```

La chiave privata è generata come valore casuale tra 2 e $p - 2$ e con la lunghezza dei bit specificata nella generazione di p (molte prove sono state eseguite con chiavi di 256 o 512 bit). La chiave pubblica si ottiene invece elevando il generatore g alla chiave privata, che viene così nascosta tramite la difficoltà del problema del logaritmo discreto.

Con `startCommunication()` si inserisce il primo messaggio di Hello nella coda, per chiedere al verifier l'inizio di un'interazione. A questo messaggio vengono associati l'identificativo e la chiave pubblica del prover da cui è generato. Una volta ricevuto dal verifier, quest'ultimo invia una challenge che, sempre passando per la coda dei messaggi e per il thread, viene gestita dal metodo `receiveChallenge(VerifierChallengeMessage message)`, che copia la chiave pubblica del verifier inviata insieme alla sfida e poi invoca `prove(message)`, eseguendo un upcasting per il messaggio da `VerifierChallengeMessage` a `ZKProverInput`. Le parti di interesse sono le seguenti, in cui viene eseguito un downcasting esplicito e vengono calcolati i valori richiesti dal verifier per la verifica:

```

1     VerifierChallengeMessage message = (VerifierChallengeMessage)
        arg0;
2     challenge = message.getChallenge();
3
4     BigInteger randomR = generateRandomNumber(P);
5     BigInteger y1 = G.modPow(randomR, P);
6     BigInteger y2 = verifierPublicKey.modPow(randomR, P);
7     BigInteger z =
        randomR.add(privateKey.multiply(challenge)).mod(phi);

```

Infine, con le informazioni, viene creato il messaggio contenente la prova.

4.3.3 Verifier

Nell'applicazione di questo schema si vuole che ci sia un unico verifier e questo è garantito applicando il Singleton pattern. In questo pattern è presente un metodo `getInstance()` pubblico e statico che ha il compito di ritornare l'unica istanza della classe, se presente, o chiamare il costruttore privato e ritornare quella appena creata, altrimenti.

Per gestire le informazioni associate ai vari provers, si è optato per l'utilizzo di una mappa basata sugli UUID dei provers. Ogni UUID è associato a un oggetto della classe di supporto `ProverInfo`, che memorizza le informazioni relative alla chiave pubblica, alla challenge e al segreto condiviso necessario per la verifica.

Con la creazione del verifier vengono create le chiavi con il metodo privato `generateKeys()` in maniera del tutto analoga a quanto visto per i provers.

Quando si riceve un messaggio da parte di un prover che vuole iniziare una nuova comunicazione, il verifier controlla che l'UUID non sia già presente nella mappa e in caso contrario viene sollevata un'eccezione. La volontà di tenere traccia dei provers con cui si sta comunicando, oltre che per memorizzare le relative informazioni in maniera efficiente, è motivata dal voler evitare che il prover cambi la sua chiave una volta ricevuta la challenge. L'invio della chiave avviene infatti proprio con il

messaggio per richiedere un'interazione e mandandone un altro si potrebbe sovrascrivere la chiave pubblica con un nuovo valore o farsi generare una nuova sfida dal verifier. Per questo motivo, ogni interazione tra una coppia prover-verifier deve essere necessariamente portata a termine esplicitamente, con un'approvazione o un rifiuto, prima di lasciar interagire di nuovo le stesse due parti. Tuttavia, se il messaggio iniziale è un primo messaggio valido per un prover non presente nella mappa, si procede con la generazione della sfida con il metodo privato *generateChallenge()*, che ritorna il valore

```
1    return new BigInteger(P.bitLength(), new
        SecureRandom()).mod(P.subtract(BigInteger.ONE)).add(BigInteger.ONE);
```

si ottiene il segreto condiviso con il metodo privato

```
1    private BigInteger getSharedSecret(BigInteger proverPublicKey) {
2        return proverPublicKey.modPow(privateKey, P);
3    }
```

che eleva la chiave pubblica del prover alla propria chiave privata, per ottenere $(g^a)^b$, e infine si aggiungono le informazioni all'interno della mappa e si invia il messaggio

```
1    proverInfoMap.put(proverUniqueID, new
        ProverInfo(proverPublicKey, challenge, sharedSecret));
2    DDHMain.insertVerifierMessage(new
        VerifierChallengeMessage(MessageType.MESSAGE_CHALLENGE, challenge,
        publicKey, proverUniqueID));
```

Quando infine si riceve una prova, il prover viene rimosso dalla mappa per permettere una nuova interazione futura e prima di inviare il messaggio di terminazione con l'esito, si esegue l'operazione di verifica con il metodo *verify(ZKCommonInput arg0)*, a seguito sempre di un'operazione di upcasting e di una di downcasting come per il metodo *prove()*. In questo metodo, dopo aver recuperato le informazioni necessarie sia dalla mappa con le informazioni del prover che dal messaggio, si passa alla verifica dei due controlli richiesti dal DDH, ritornando infine un booleano per segnalare se la sfida risulta superata o meno.

```
1    public boolean verify(ZKCommonInput arg0) {
2
3        ProverProofMessage message = (ProverProofMessage) arg0;
4        UUID proverUniqueID = message.getProverUniqueID();
5        ProverInfo proverInfo = proverInfoMap.get(proverUniqueID);
6
7        BigInteger y1 = message.getY1();
8        BigInteger y2 = message.getY2();
9        BigInteger z = message.getZ();
10
11        boolean firstControl = G.modPow(z,
            P).equals(proverInfo.getProverPublicKey().modPow(
            proverInfo.getChallenge(), P).multiply(y1).mod(P));
12        boolean secondControl =
            publicKey.modPow(z, P).equals(proverInfo.getSharedSecret()
            .modPow(proverInfo.getChallenge(),
            P).multiply(y2).mod(P));
13
14        return firstControl && secondControl;
15
16    }
17
```

4.3.4 Osservazioni finali

Da un'osservazione della struttura del progetto, si possono notare somiglianze anche con i Sigma-Protocols, tra cui le principali:

1. è presente una fase di interazione tra le parti;
2. non si tratta di un'implementazione di un protocollo laconico, nè per numero di round, nè come risposta del prover che non solo non invia un singolo bit (invia un `BigInteger`), ma invia tre valori distinti y_1 , y_2 e z ;
3. sono presenti e chiaramente distinguibili le tre fasi di inizializzazione, interazione e conclusione, e in particolare la fase di interazione si svolge seguendo lo schema Prover-Verifier-Prover illustrato nel capitolo 2 (si noti come l'ultimo messaggio di terminazione del Verifier è in realtà parte della fase conclusiva, non di interazione, e potrebbe comunque essere omesso in caso di gestione alternativa del responso del verificatore);
4. infine, seppure l'incapsulamento e la struttura stessa dei messaggi non permettano al Verifier di scoprire informazioni aggiuntive, questo non basta per affermare che il protocollo rispetti a pieno titolo la definizione di ZK adatta anche a verificatori potenzialmente non onesti, in quanto in questo caso il Verifier è stato esplicitamente progettato per essere onesto. Per affermare la piena proprietà ZK andrebbe costruita una dimostrazione formale e, in mancanza di quest'ultima, è possibile ricadere solo nella categoria di HVZK, in quanto comunque si esegue una prova di conoscenza senza lo scambio effettivo del segreto o di informazioni sensibili.

La fase di implementazione pratica è stata completata con la generazione della documentazione Javadoc, di cui si riporta la gerarchia nell'Appendice D, e con la creazione di un diagramma simil-UML delle classi [5], di cui si riporta in Appendice C una versione semplificata e in cui si omette la gerarchia dei messaggi discussa in precedenza, per favorire la leggibilità.

Capitolo 5

Conclusioni

Il vasto panorama di protocolli interattivi introdotto, porta a considerazioni sul punto in cui ci si trova attualmente e sui benefici che lo studio di tale ambito può ancora portare. I protocolli ZK sono protocolli ormai consolidati e di indubbia valenza, che trovano già notevoli applicazioni negli usi comuni dei protocolli di autenticazione e di verifica della conoscenza, mentre si trovano ancora in una fase emergente i PAoK. Oltre agli esempi di applicazioni pratiche per i protocolli interattivi (e in particolare basati su ZK) suggeriti nell'introduzione di questa relazione di tirocinio, è possibile suggerire un altro scenario tra quelli notevoli: il loro utilizzo nelle blockchain e nelle dark pools (piattaforme private di trading), con dimostrazioni di conoscenza di una transazione senza rivelare l'ammontare specifico o le parti coinvolte. Per esempio, una possibile finalità è quella di dimostrare di avere abbastanza fondi per effettuare una transazione, senza dover rivelare l'importo esatto posseduto dal mittente, in modo da mantenere la privacy senza esporre dettagli sensibili (particolarmente utile in questo caso l'uso di protocolli ZK-SNARK o la considerazione già discussa sull'equivalenza tra PAoK e WE).

Dal punto di vista teorico, la domanda lasciata aperta da [FNV17] circa la possibilità di costruire WI-PA (Witness Indistinguishable Predictable Arguments), è stata affrontata recentemente da [Kho22], che ha risposto mediante l'uso di dimostrazioni witness-indistinguishable non interattive, lasciando a sua volta aperto un altro quesito circa la possibilità o meno di dimostrare che WI Deterministic-Prover Argument implichi WI-PA.

Concentrandosi invece sulla parte applicativa, è stata discussa l'importanza di avere a disposizione librerie come SCAPI incentrate sulla computazione sicura e che mettono insieme diverse primitive crittografiche utili allo scopo. Nonostante la sua utilità, il Bar Ilan Cryptography Research Group sembra aver messo da parte il supporto alla libreria a favore di libscapi [6], una libreria crittografica open-source del tutto simile a SCAPI, ma in linguaggio C++. Date le caratteristiche di Java e le modalità con cui si adatta bene ai ruoli di protocolli ZK e simili, con la possibilità di avere classi specifiche e di sfruttare le caratteristiche offerte dalla programmazione a oggetti, sarebbe utile avere in futuro anche altre librerie con le stesse qualità offerte da quella presentata in queste pagine, che magari integri le interfacce astratte con varianti anche solo leggermente più concrete, in modo da non vincolare eccessivamente nelle implementazioni, ma allo stesso tempo fornire più segnature di metodi come strumenti di appoggio su cui poter fare affidamento, sfruttando eventualmente anche l'overloading dei metodi in maniera opportuna.

Come sviluppi futuri si potrebbe pensare all'utilizzo di SCAPI in circostanze più pratiche e in ambienti distribuiti in cui, per esempio, poter prevedere l'utilizzo

del `CommunicationLayer` messo a disposizione per la creazione di canali socket, utilizzando il DDH implementato come tassello per protocolli più articolati come per esempio protocolli di verifica dell'identità per autenticazione o nella crittografia a chiave pubblica.

Per concludere, l'attività di tirocinio mi ha dato l'opportunità di avvicinarmi con un approccio per la prima volta più pratico all'ambito della crittografia e più in particolare degli schemi interattivi. Le sfide principali che si sono presentate sono state sicuramente legate alla comprensione dei concetti avanzati necessari per comprendere sia alcune parti delle dimostrazioni di correttezza degli argomenti trattati, che le interconnessioni presenti tra le varie primitive e i vari protocolli crittografici, ma anche all'implementazione concreta mediante interfacce astratte di argomenti trattati finora solo dal punto di vista concettuale. Un aspetto che reputo personalmente di estrema importanza, per questo tirocinio e per questa relazione, è la valenza dell'approccio pratico per approfondire e consolidare i concetti teorici, in quanto mi ha permesso di mettermi alla prova andando a toccare con mano e da vicino degli aspetti che sono di fondamentale e attuale importanza per garantire la sicurezza in molteplici ambiti.

Appendice

A Terminologia

Di seguito viene presentato un breve compendio di parte della terminologia usata in questa trattazione.

Argument of Knowledge:

un protocollo interattivo utilizzato in crittografia e teoria della computazione. Concetto legato e contrapposto a quello di Proof of Knowledge. Questo protocollo coinvolge un "prover" e un "verifier", dove il prover cerca di dimostrare di possedere una certa conoscenza senza rivelare direttamente tale informazione. L'obiettivo è convincere il verifier della conoscenza specifica senza svelare dettagli sensibili. L'Argument of Knowledge è progettato in modo tale che, se il prover supera con successo il protocollo, il verifier sia convinto della conoscenza del prover senza tuttavia acquisire informazioni dettagliate. Oltre a dimostrare la conoscenza, l'AoK può coinvolgere una discussione più ampia e interattiva che potrebbe includere l'interscambio di ulteriori informazioni senza rivelazioni complete, quindi può essere utilizzato in scenari in cui la dimostrazione di conoscenza non è l'unico obiettivo.

Commitment:

impegno o vincolo. Nel contesto dei protocolli crittografici, un "commitment" rappresenta un meccanismo mediante il quale una parte può vincolarsi in modo irrevocabile a un particolare valore o informazione, senza divulgarlo immediatamente. L'impegnante associa il valore a un impegno crittografico, che può essere visto come una rappresentazione cifrata e crittograficamente sicura dello stesso. Tale concetto trova applicazione in situazioni in cui è necessario impegnarsi preventivamente su una scelta o su un dato, mantenendo segreta la sua natura fino a un momento successivo, quando la parte decide di rivelare parzialmente o completamente le informazioni durante una fase di *decommitment*, per dimostrare di aver impegnato correttamente il valore.

Extractable Witness Encryption:

aggiunge un'ulteriore proprietà al Witness Encryption Scheme. Non solo chi possiede la witness può decifrare il messaggio, ma è anche possibile estrarre una testimonianza da un testo cifrato (ciphertext) senza conoscere il messaggio originale. Se si dispone del messaggio cifrato e si ha un modo per estrarre una witness w da quel cifrato, è possibile utilizzare tale w per dimostrare che il messaggio ha una particolare proprietà o soddisfa determinati criteri, senza rivelare il messaggio effettivo o la witness stessa.

Moneta virtuale (Coin):

un generatore di numeri casuali che può assumere valori 0 o 1. L'analogia si basa

sul classico lancio di una moneta, dove si ha la possibilità di ottenere uno dei due risultati possibili: testa o croce. Si distinguono in *private coins*, il cui risultato non è condiviso con l'altra parte (solitamente il Prover in questi contesti), e *public coins*, in cui il risultato è invece accessibile anche all'esterno.

Oracolo:

termine usato per riferirsi a un dispositivo teorico che può essere consultato da un algoritmo o da un agente per rispondere a domande relative a un problema specifico. L'oracolo ha una conoscenza speciale o un accesso privilegiato a un certo insieme di dati e informazioni.

Perfectly binding (perfettamente vincolante):

caratteristica di un protocollo di impegno crittografico per cui, una volta che un determinato valore è stato vincolato, non può essere modificato o sostituito senza che ciò venga rilevato. Questa proprietà assicura che il committente non possa alterare il valore, dopo averlo impegnato, senza subire conseguenze.

Perfectly hiding:

caratteristica di un protocollo di impegno crittografico per cui il valore impegnato è completamente nascosto, ovvero non può essere dedotto o determinato da osservatori esterni. Il valore è segreto e non rivelato fino alla fase di *decommitment* (rivelazione).

Primitiva:

una funzione, un algoritmo o un protocollo che viene utilizzato come elemento fondamentale per costruire sistemi crittografici più complessi. L'implementazione congiunta di diverse primitive consente la creazione di sistemi crittografici robusti e diversificati.

Proof of Knowledge:

una categoria specifica di Argument of Knowledge. In un protocollo di Proof of Knowledge, il prover dimostra in modo convincente al verifier di possedere una certa conoscenza senza rivelare direttamente l'informazione. Questa dimostrazione segue specifiche proprietà, come correttezza e affidabilità. L'obiettivo principale di una Proof of Knowledge è fornire una prova convincente senza rivelare dettagli sensibili, contribuendo così alla progettazione di protocolli crittografici sicuri. La prova è al centro dell'attenzione, dimostrando che una conoscenza segreta esiste senza rivelarla direttamente. Per questo motivo, viene spesso utilizzata per dimostrare la conoscenza di una chiave privata senza rivelarla.

Trapdoor function:

una funzione facile da computare in una direzione, ma difficile da calcolare in quella opposta se non si possiedono determinate informazioni.

Witness:

testimonianza; è un'informazione, o un insieme di informazioni aggiuntive, di natura riservata che permette al Prover di comprovare la veridicità di un'affermazione (statement).

Witness Encryption Scheme:

una primitiva che consente la crittografia di un messaggio in modo che solo chi possiede una witness possa decifrarlo e recuperarlo.

B Acronimi

Alcuni tra i principali acronimi utilizzati.

AM Arthur Merlin proof.

CRS Common Reference String model.

DDH Decisional Diffie–Hellman assumption.

Ext-WE Extractable Witness Encryption.

HVZK Honest-Verifier Zero-Knowledge.

NIZK-PoK Non-Interactive Zero-Knowledge Proof of Knowledge.

NP Nondeterministic Polynomial Time.

OT Oblivious Transfer.

P Polynomial Time o Prover, a seconda dei contesti in cui viene utilizzato.

PAoK Predictable Arguments of Knowledge.

PPT Probabilistic Polynomial Time.

PZAP Predictable ZAP.

V Verifier.

WE Witness Encryption.

WI-PA Witness Indistinguishable Predictable Arguments.

ZK Zero-Knowledge.

ZK-CPPA Zero-Knowledge Commit-and-Prove Predictable Arguments.

ZKP Zero-Knowledge Proofs.

ZK-PAoK Zero-Knowledge Predictable Argument of Knowledge.

ZK-PoK Zero-Knowledge Proofs of Knowledge.

C Diagramma delle classi

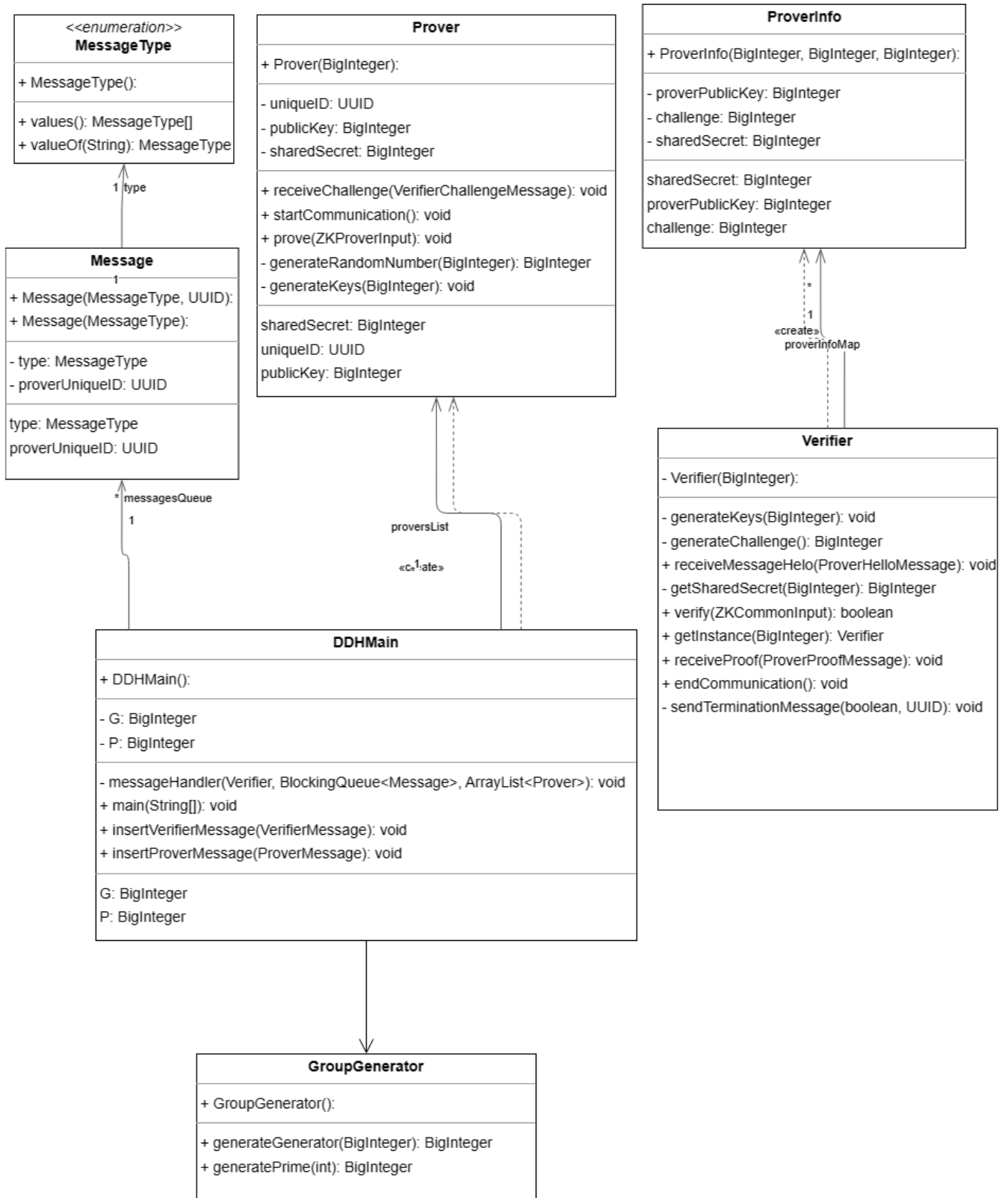


Figura 5.1. Diagramma delle classi

D Gerarchia delle classi Javadoc

Hierarchy For Package ddh_zk

Class Hierarchy

- java.lang.**Object**[↗]
 - ddh_zk.**DDHDemo**
 - ddh_zk.**GroupGenerator**
 - ddh_zk.**Message** (implements edu.biu.scapi.interactiveMidProtocols.zeroKnowledge.ZKCommonInput)
 - ddh_zk.**ProverMessage**
 - ddh_zk.**ProverHelloMessage**
 - ddh_zk.**ProverProofMessage** (implements edu.biu.scapi.interactiveMidProtocols.zeroKnowledge.ZKCommonInput)
 - ddh_zk.**VerifierMessage**
 - ddh_zk.**VerifierChallengeMessage** (implements edu.biu.scapi.interactiveMidProtocols.zeroKnowledge.ZKProverInput)
 - ddh_zk.**VerifierEndCommMessage**
 - ddh_zk.**VerifierTerminationMessage**
 - ddh_zk.**Prover** (implements edu.biu.scapi.interactiveMidProtocols.zeroKnowledge.ZKProver)
 - ddh_zk.**ProverInfo**
 - java.lang.**Throwable**[↗] (implements java.io.Serializable[↗])
 - java.lang.**Exception**[↗]
 - ddh_zk.**DuplicatedProverException**
 - ddh_zk.**Verifier** (implements edu.biu.scapi.interactiveMidProtocols.zeroKnowledge.ZKVerifier)

Enum Class Hierarchy

- java.lang.**Object**[↗]
 - java.lang.**Enum**[↗]<E> (implements java.lang.Comparable[↗]<T>, java.lang.constant.Constable[↗], java.io.Serializable[↗])
 - ddh_zk.**MessageType**

Figura 5.2. Gerarchia delle classi generata con la documentazione Javadoc

Bibliografia

- [1] Bouncy Castle: <https://www.bouncycastle.org/>
- [2] Immagine architettura SCAPI: <https://scapi.readthedocs.io/en/latest/intro.html>
- [3] Immagine livelli di sicurezza SCAPI: https://scapi.readthedocs.io/en/latest/primitives/security_levels.html
- [4] Immagine realizzata con l'uso di Canva: <https://www.canva.com>
- [5] Immagine realizzata con l'uso di draw.io (diagrams.net): <https://www.drawio.com>
- [6] libscapi, pagina Github del progetto: <https://github.com/cryptobiu/libscapi>
- [7] OpenSSL: <https://www.openssl.org/>
- [8] Professor Yehuda Lindell: <https://cris.biu.ac.il/en/persons/yehuda-lindell>
- [9] SCAPI documentation: <https://scapi.readthedocs.io/en/latest/index.html>
- [10] SCAPI license:
 - Sito ufficiale di SCAPI: <http://crypto.biu.ac.il/scapi>
 - Licenza d'uso di SCAPI: <https://scapi.readthedocs.io/en/latest/license.html>
- [AB07] Sanjeev Arora e Boaz Barak. Computational Complexity: A Modern Approach. Cambridge University Press, 2009.
- [BIN97] Mihir Bellare, Russell Impagliazzo e Moni Naor. Does parallel repetition lower the error in computationally sound protocols?. In FOCS, pagine 374–383, 1997.
- [BM88] László Babai e Shlomo Moran. Arthur-Merlin games: A randomized proof system, and a hierarchy of complexity classes. In Journal of Computer and System Sciences, volume 36, issue 2, pagine 254-276, 1988.
- [BSMP91] Manuel Blum, Alfredo De Santis, Silvio Micali e Giuseppe Persiano. Noninteractive zero-knowledge. In SIAM Journal on Computing, 1991.
- [FNV17] Antonio Faonio, Jesper Buus Nielsen e Daniele Venturi. Predictable arguments of knowledge. In Serge Fehr, editor, PKC 2017, Part I, volume 10174 of LNCS, pages 121–150. Springer, Heidelberg, 2017.
- [GK96] Oded Goldreich and Ariel Kahan. How To Construct Constant-Round Zero-Knowledge Proof Systems for NP. In Journal of Cryptology, pagine 167–190, 1996.

- [GMR85] Shafi Goldwasser, Silvio Micali e Charles Rackoff. The knowledge complexity of interactive proof-systems. In STOC '85: Proceedings of the seventeenth annual ACM symposium on Theory of computing, pagine 291–304, 1985.
- [Kho22] Hamidreza Khoshakhlagh. (Commit-and-Prove) Predictable Arguments with Privacy. In: Ateniese, G., Venturi, D. (eds) Applied Cryptography and Network Security. ACNS 2022. Lecture Notes in Computer Science, vol 13269. Springer, Cham. 2022.
- [Lin21] Yehuda Lindell. How To Simulate It - A Tutorial on the Simulation Proof Technique, sezione 5, pagine 14-31. 2021.
- [PW12] Krzysztof Pietrzak and Douglas Wikström. Parallel repetition of computationally sound protocols revisited. J. Cryptology, pagine 116–135, 2012.
- [SH92] Adi Shamir. $IP = PSPACE$. In Journal of the ACM, volume 39, issue 4, pagine 869–877s, 1992.

Ringraziamenti

Questa relazione di tirocinio rappresenta per me la conclusione di un percorso importante e l'inizio di uno nuovo. In queste righe finali, desidero esprimere la mia gratitudine a tutte le persone (e non) che hanno fatto parte del mio percorso.

Innanzitutto, vorrei ringraziare il mio relatore, il Professor Daniele Venturi, per la sua guida preziosa nel progetto di tirocinio. La sua disponibilità e il suo supporto sono stati fondamentali, fornendomi chiarimenti e consigli che hanno arricchito la mia esperienza.

Un sentito ringraziamento va ai miei genitori, che con non pochi sacrifici hanno reso possibile il mio percorso universitario, anche se questo ha comportato l'avermi lontana da casa, e alle mie sorelle, per essermi sempre state vicine. E' con il supporto della mia famiglia che ho avuto la possibilità di fare così tante esperienze stupende.

Ringrazio il mio ragazzo Simone, per la sua presenza e il suo sostegno costanti, che mi hanno motivato e che hanno reso questo percorso ancora più significativo. Sono grata di avere una persona così straordinaria al mio fianco. Ringrazio anche la sua famiglia, per l'accoglienza e il calore che mi hanno fatto sentire a casa anche qui a Roma.

Non posso dimenticare gli amici, in particolare Luca, che ha sopportato anche e non solo i miei esaurimenti notturni prima degli esami più difficili e che c'è sempre stato come migliore amico di una vita. Grazie di cuore a coloro che ho incontrato tra i banchi universitari, che sono diventati ben più che colleghi, e che hanno contribuito a rendere questi anni ancora più speciali.

Un grazie anche a me stessa, per la determinazione nel raggiungere questo traguardo, e un ringraziamento speciale all'amico che mi ha spronato a seguire i miei sogni prima della scelta dell'università.

Infine, un ringraziamento affettuoso va a Mami, gattina del Dipartimento, mascotte e portafortuna degli studenti di Informatica, per aver reso più leggeri i momenti di tensione con le sue fusa.